

Aprendizado de Máquina Supervisionado –IMD3002

Aula 18 – Redes Neurais Artificias

João Carlos Xavier Júnior

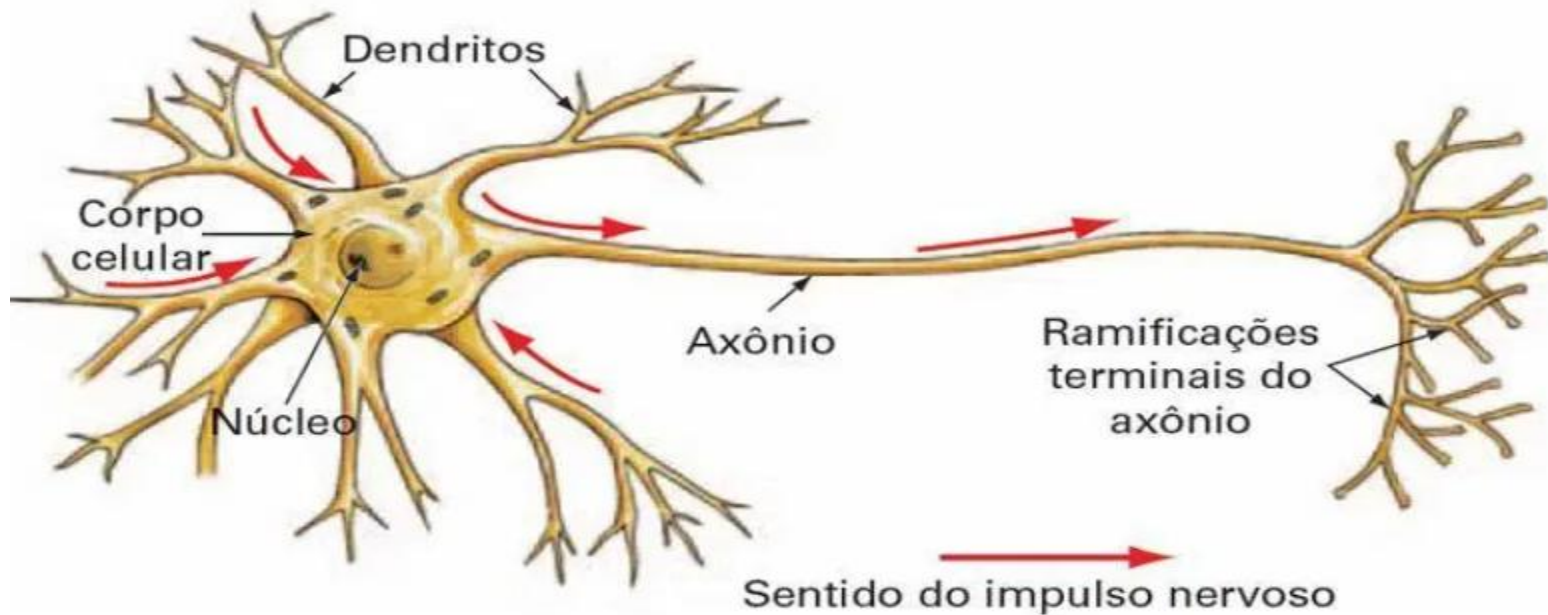
jcxavier@imd.ufrn.br

Redes Neurais Artificiais

Redes Neurais Biológicas

❑ Cérebro humano e seus componentes:

❖ Neurônio Biológico:

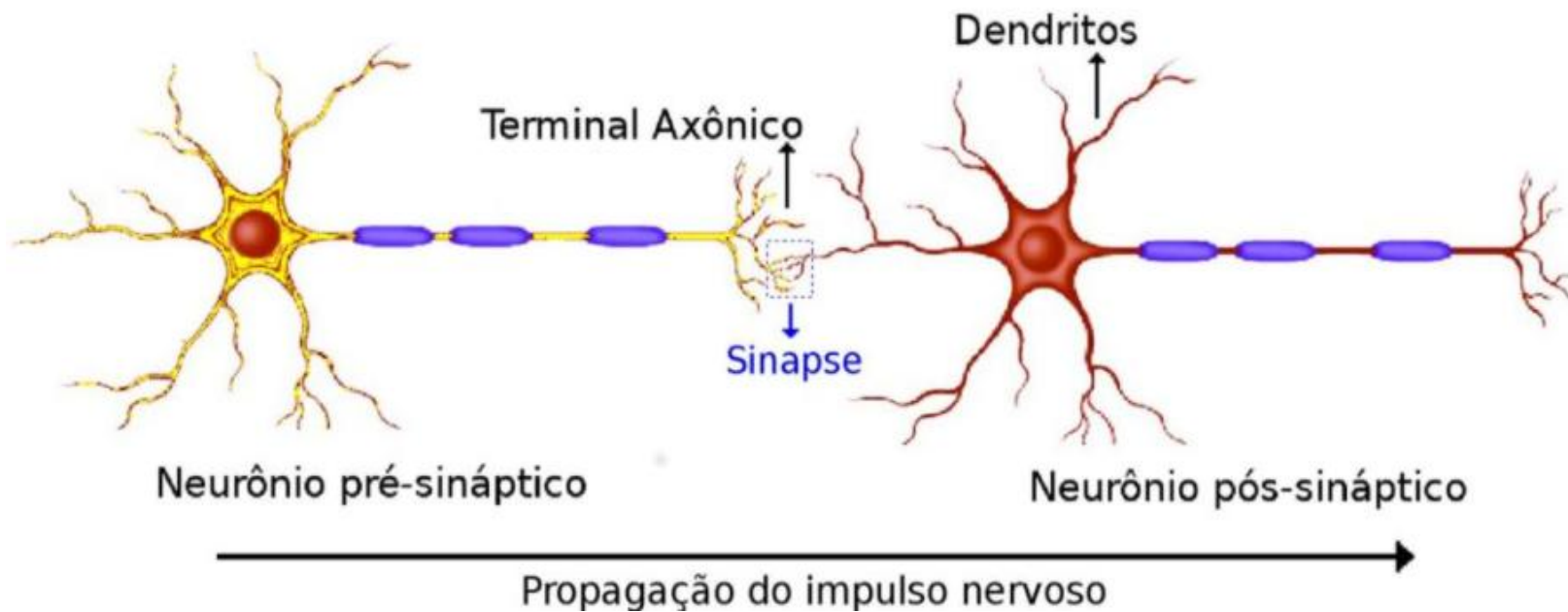


<http://deeplearningbook.com.br/o-neuronio-biologico-e-matematico/>

Redes Neurais Biológicas

❑ Cérebro humano e seus componentes:

❖ Sinapses:



<https://escolaeducacao.com.br/sinapses/>

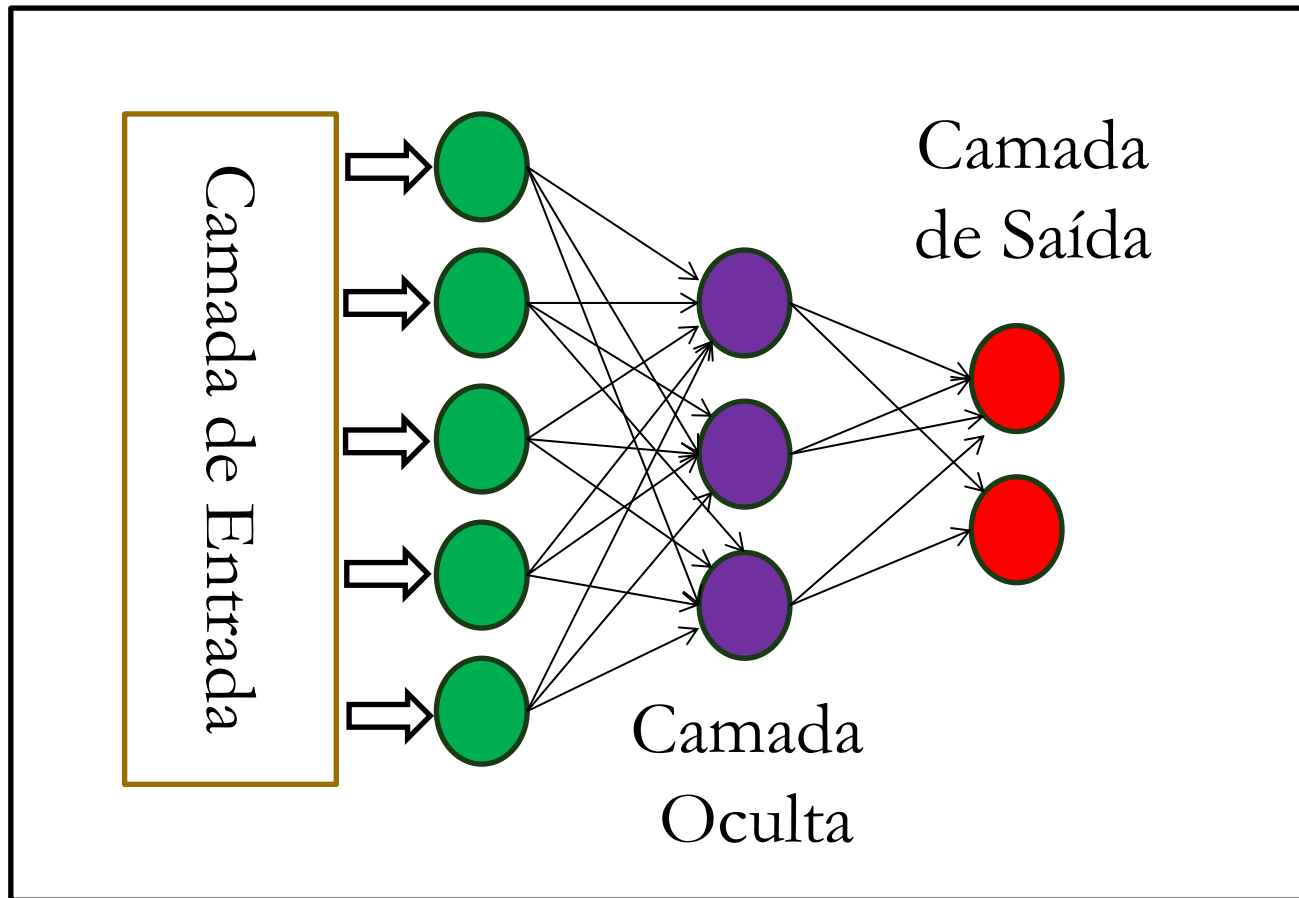
Redes Neurais Artificiais

- ❑ **Modelo Matemático inspirado no cérebro humano:**
 - ❖ Composto por várias unidades de processamento (“neurônios”);
 - ❖ Interligadas por um grande número de conexões (“sinapses”).

Redes Neurais Artificiais

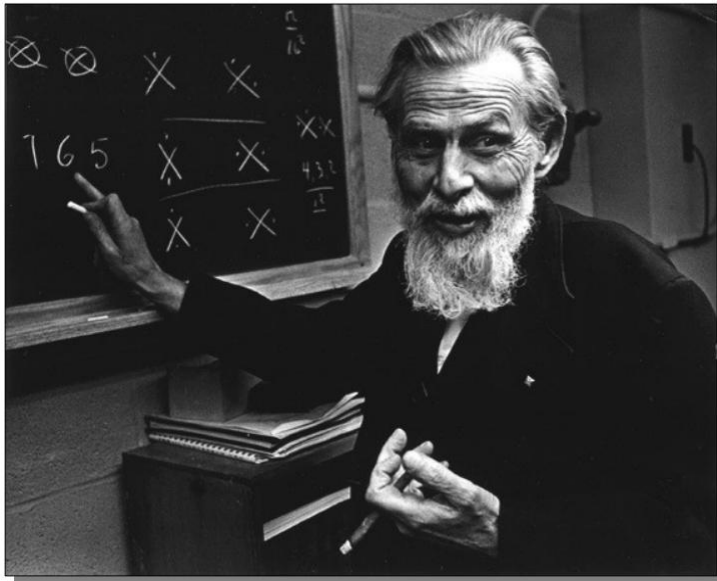
- ❑ Redes Neurais Artificiais (RNA) são modelos de computação com propriedades particulares:
 - ❖ Capacidade de se adaptar ou aprender;
 - ❖ Generalizar;
 - ❖ Agrupar ou organizar dados.

Estrutura de uma Rede Neural

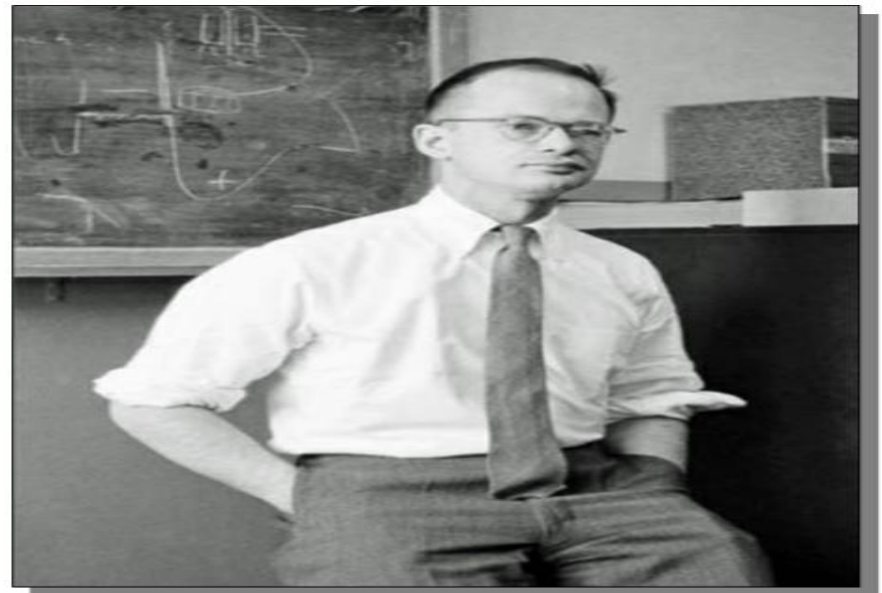


Neurônio Artificial

- A primeira noção de neurônio artificial foi proposta em 1943 por dois sujeitos: **Warren McCulloch** e **Walter Pitts**.



McCulloch:
Psiquiatra e Neuroanatomista



Pitts:
Matemático

Neurônio de McCulloch e Pitts (MP)

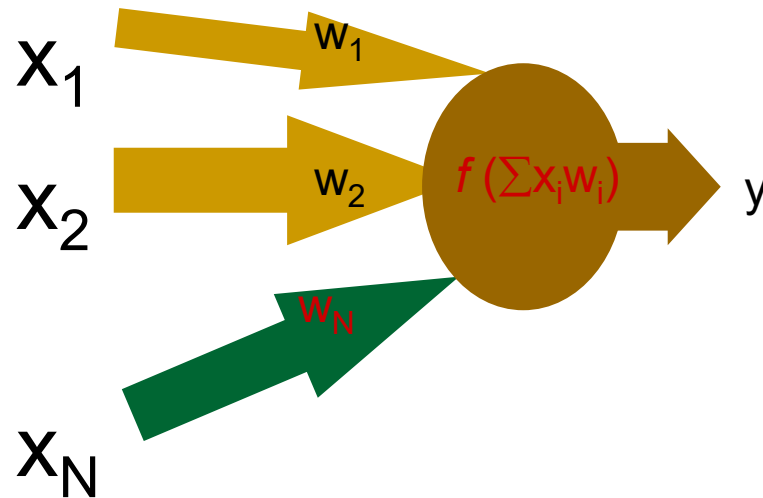
- ❑ Modelo era uma **simplificação** do que se sabia na época sobre o neurônio biológico.
- ❑ Sua representação matemática resultou em um modelo com:
 - ❖ N valores de entradas (referenciados com x_n);
 - ❖ Valores em suas arestas, representando sinapses (pesos, referenciados com W_N);
 - ❖ Apenas 1 (um) valor de saída (referenciado com y).
- ❑ Os pesos são multiplicados às entradas: $x_i w_i$.

Neurônio de McCulloch e Pitts (MP)

□ Entrada total:

$$u = \sum_{j=1}^N x_j w_j$$

↓
Soma ponderada

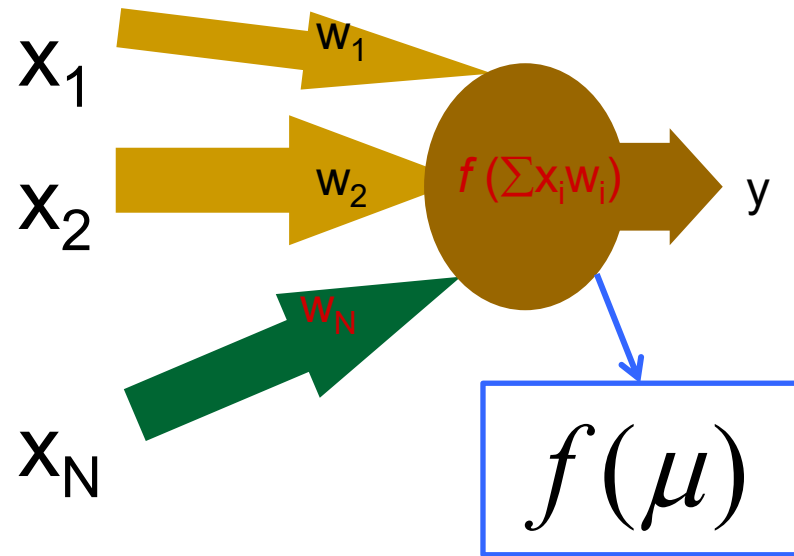


Neurônio de McCulloch e Pitts (MP)

□ Entrada total:

$$u = \sum_{j=1}^N x_j w_j$$

Soma ponderada

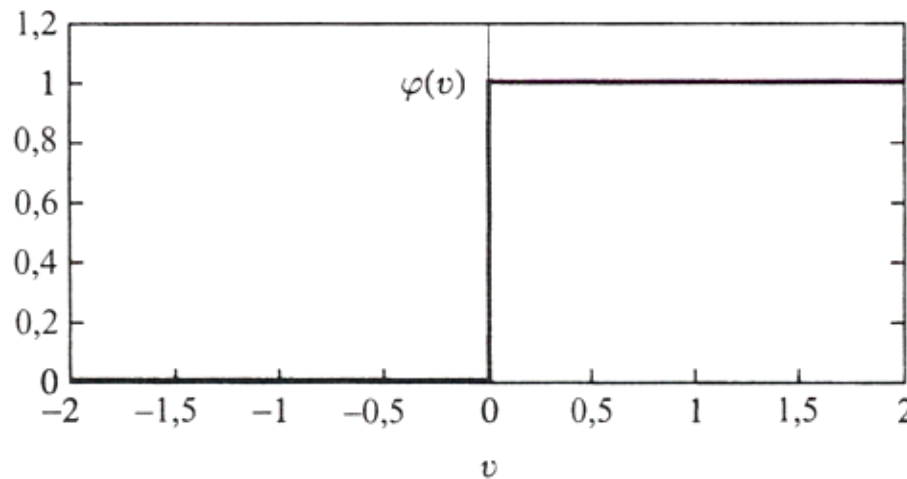


Função de ativação

Função de Ativação

□ Função Limiar ou Passo:

$$\varphi(y) = \begin{cases} 1 & \text{se } x > 0 \\ 0 & \text{se } x \leq 0 \end{cases}$$



Funções de Ativação

- Estado de ativação:
 - ❖ Representa o estado dos neurônios da rede.
 - ❖ Pode assumir valores:
 - Binários (0 e 1);
 - Bipolares (-1 e +1);
 - Reais.
 - ❖ Quais funções de ativação podem ser usadas?
 - Qualquer uma que:
 - Seja contínua;
 - Monotonicamente crescente; e
 - Que gera valores graduais no intervalo $[0,1]$.

Funções de Ativação

❑ Funções mais comuns:

❖ Linear

$$a(t + 1) = u(t)$$

❖ Limiar

$$a(t + 1) = \begin{cases} 1, & \text{se } u(t) \geq \theta \\ 0, & \text{se } u(t) < \theta \end{cases}$$

❖ Sigmóide

$$a(t + 1) = 1/(1 + e^{-\lambda u(t)})$$

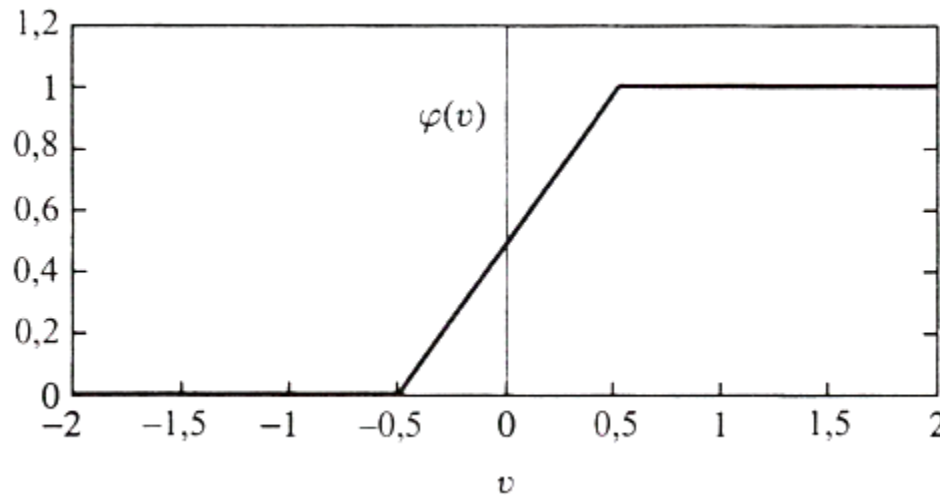
❖ Tangente Hiperbólica

$$a(t + 1) = \frac{(1 - e^{-\lambda u(t)})}{(1 + e^{-\lambda u(t)})}$$

Funções de Ativação

❑ Função Linear:

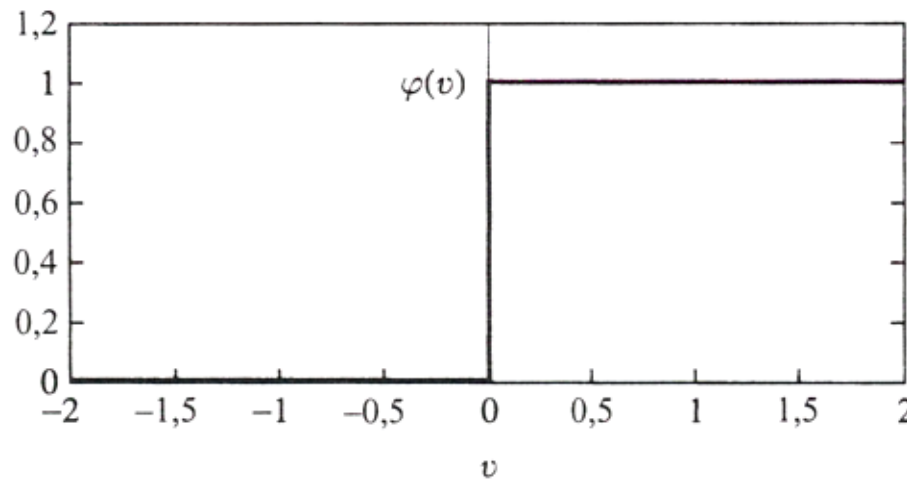
$$\varphi(y) = \begin{cases} 1 & \text{se } x \geq +1/2 \\ y & \text{se } 1/2 > x > -1/2 \\ 0 & \text{se } x \leq -1/2 \end{cases}$$



Função de Ativação

□ Função Limiar:

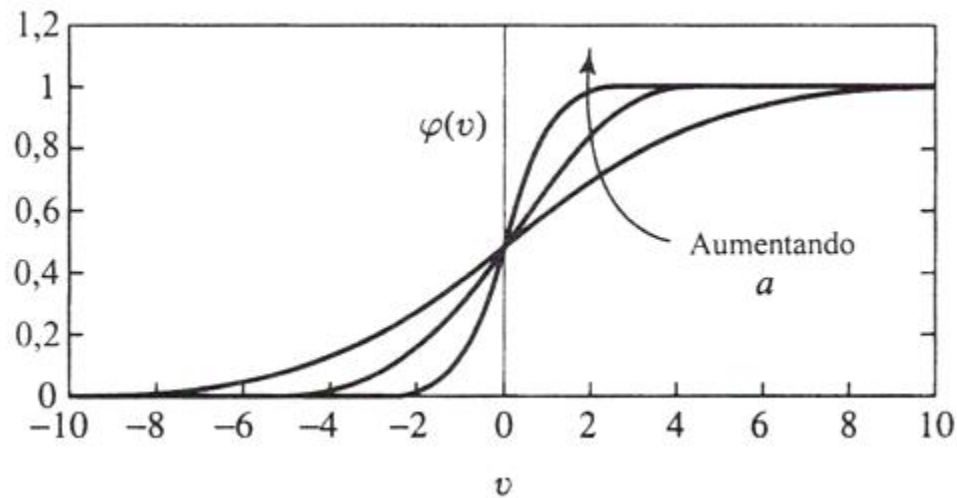
$$\varphi(y) = \begin{cases} 1 & \text{se } x > 0 \\ 0 & \text{se } x \leq 0 \end{cases}$$



Funções de ativação

❑ Função Sigmóide:

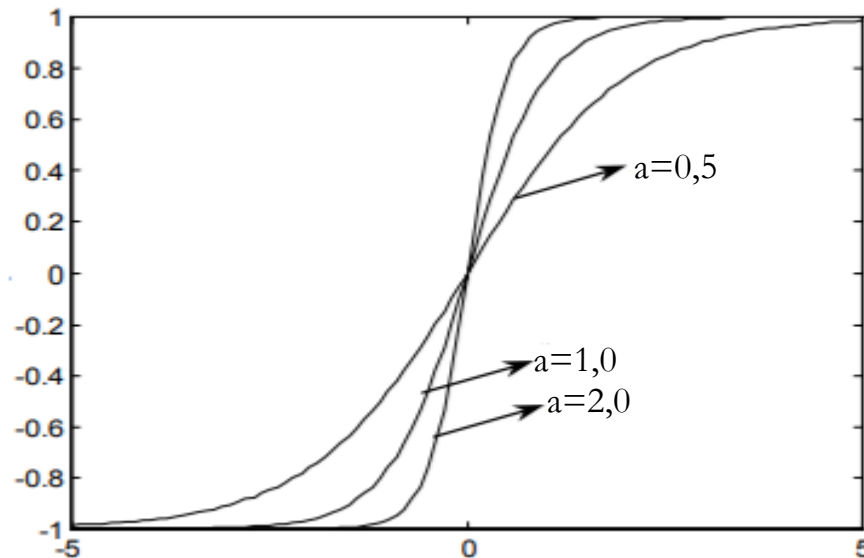
$$\varphi(y) = \frac{1}{1 + e^{(-ax)}} \quad \text{a é o parâmetro da inclinação}$$



Funções de ativação

❑ Função Tangente Hiperbólica:

$$\varphi(y) = \frac{1 - e^{(-ax)}}{1 + e^{(-ax)}}$$



Conexões

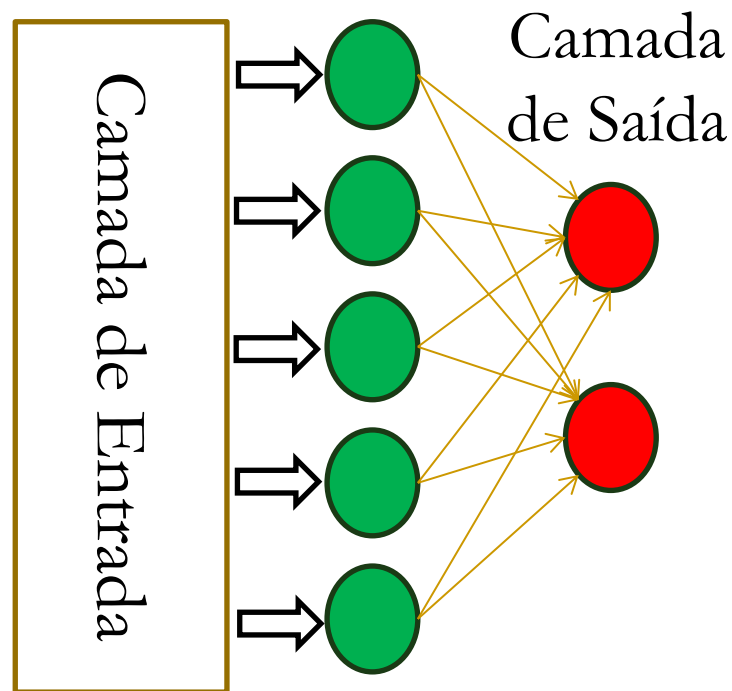
- ❑ Definem como os neurônios estão interligados.
 - ❖ Nós são conectados entre si através de conexões específicas.
- ❑ Uma conexão geralmente tem um valor de ponderamento ou **peso** associada a ela
- ❑ Tipos de conexões ($w_{ik}(t)$):
 - ❖ Excitatória: ($w_{ik}(t) > 0$);
 - ❖ Inibitória: ($w_{ik}(t) < 0$);
 - ❖ Conexão inexistente: ($w_{ik}(t) = 0$).

$$\begin{array}{c|c} 0 & \\ \hline -1 & +1 \end{array}$$

Topologia

□ Número de camadas:

❖ Uma camada (Ex Perceptron, Adaline)

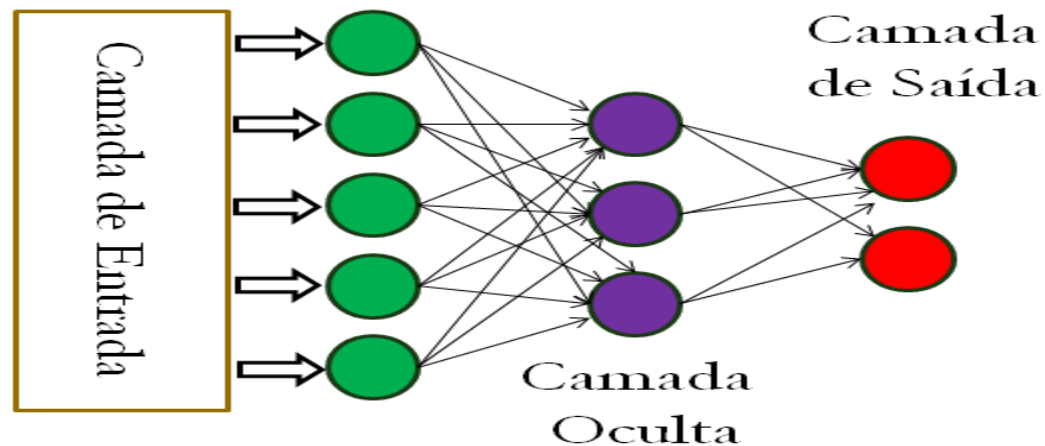


Topologia

□ Número de camadas:

❖ Multi-camadas (Ex MLP):

- Completamente conectada;
- Parcialmente conectada;
- Localmente conectada.



Topologia

□ Arranjo das conexões:

❖ Redes feedforward:

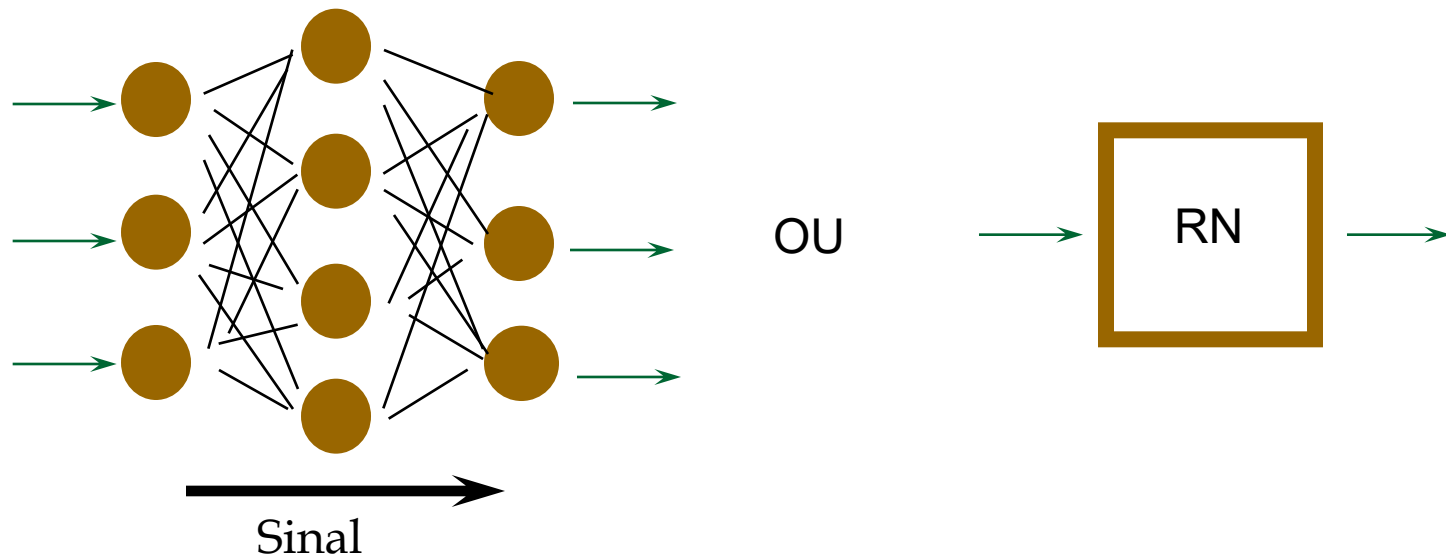
- Não existem loops de conexões.

❖ Redes recorrentes:

- Conexões apresentam loops;
- Mais utilizadas em sistemas dinâmicos.

Redes Feedforward

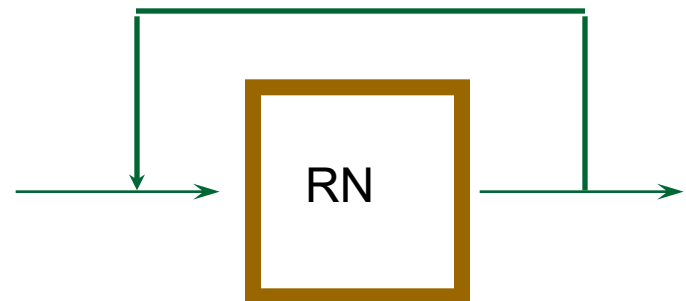
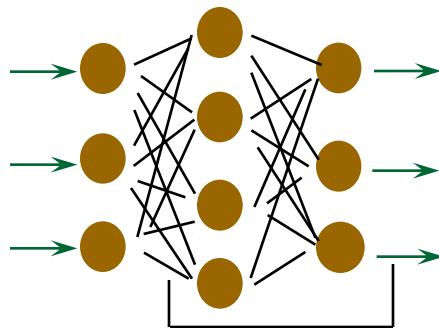
- Sinais seguem em uma única direção:



Tipo mais comum

Redes Recorrentes

- Possuem conexões ligando saída da rede a sua entrada



- Podem lembrar entradas passadas e, consequentemente, processar seqüência de informações (no tempo ou espaço).

Aprendizado em RNAs

- ❑ Capacidade de aprender a partir de seu ambiente e melhorar sua performance com o tempo.
- ❑ Parâmetros livres de uma RNA são adaptados através de estímulos fornecidos pelo ambiente:
 - ❖ Processo iterativo de ajustes aplicado a sinapses e thresholds.
 - ❖ Idealmente, a RNA sabe mais sobre seu ambiente após cada iteração.

Aprendizado em RNAs

- ❑ RNA deve produzir para cada conjunto de entradas apresentado, o conjunto de saídas desejado:

$$\diamond w_{ik}(t+1) = w_{ik}(t) + \Delta w_{ik}(t)$$

→ Atualização dos pesos

- ❑ Mecanismos de aprendizado:
 - ❖ Modificação de pesos ($\Delta w_{ij}(t)$) associados às conexões;
 - ❖ Armazenamento de novos valores em conteúdos de memória;
 - ❖ Acréscimo e/ou eliminação de conexões/neurônios.

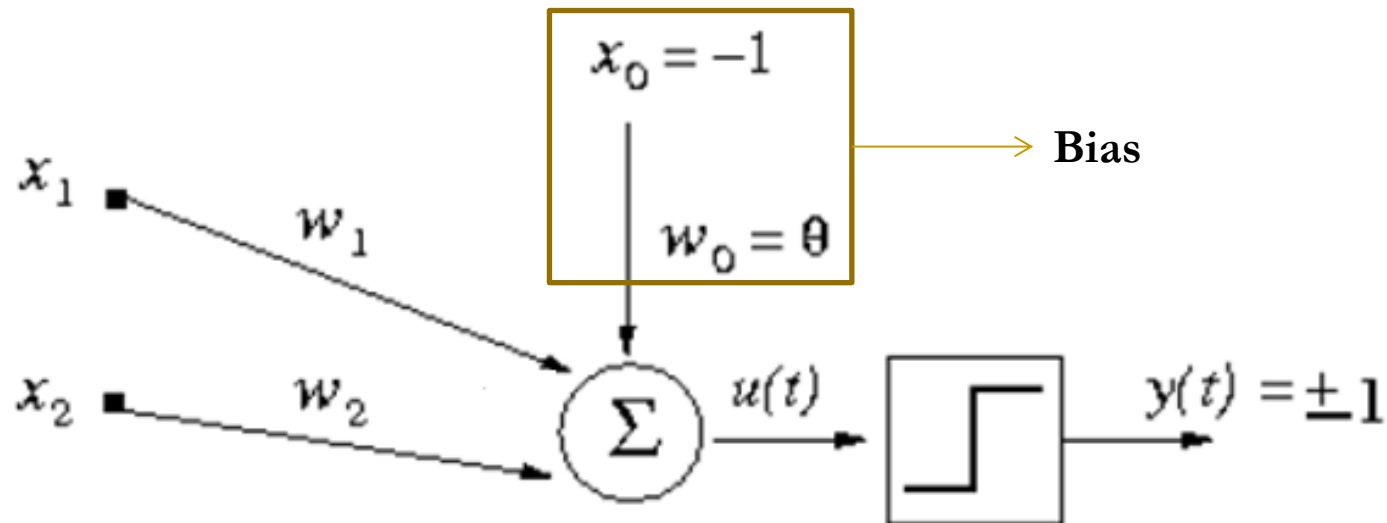
Aprendizado por Correção de Erro

- ❑ Regra Delta (Widrow e Hoff 1960)
- ❑ Erro: $e_k(t) = d_k(t) - y_k(t)$
- ❑ Minimizar função de custo baseada em $e_k(t)$
- ❑ Função de custo:
 - ❖ $c(t) = -1/2 \sum e_k^2(t)$
 - ❖ Minimização de $c(t)$ utiliza método de gradiente descendente.
 - ❖ Ajustar os pesos, visando **minimizar o erro**.
 - ❖ Aprendizado **atinge solução estável** quando os pesos não mudam muito.

Perceptrons

Perceptrons

- ❑ Desenvolvido por **Rosemblat**, 1958.
- ❑ Utiliza modelo de McCulloch-Pitts como neurônio.



Perceptrons: O papel do bias

□ O que seria **Bias**?

- ❖ Um parâmetro a mais na função de ativação;
- ❖ Valor fixo (-1, 0, ou 1);
- ❖ O uso do bias permite que definamos o valor de threshold adotado em nossa função de ativação, sendo necessário então atualizar somente os pesos e o bias na rede
- ❖ A mesma regra para atualização dos pesos é válida também para a atualização do bias.

Perceptrons: O papel do bias

- Suponha que uma função passo está sendo usada ($f(u)=1$, se $u>\theta$ e $f(u) = 0$, caso contrário)
- Sabe-se que $y=x_1*w_1+...+ x_n*w_n$; pode-se resumir o problema em:

$$x_1w_1 + x_2w_2 \geq \theta$$

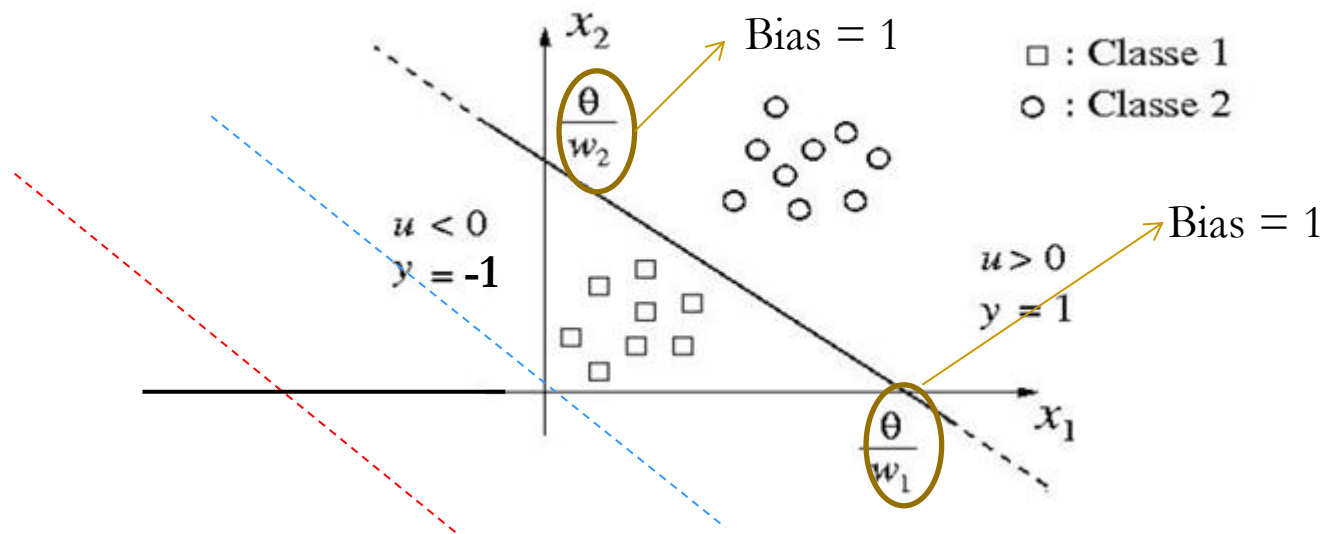
- Se a inequação for verdadeira, $f(u)=1$

Perceptrons: O papel do bias

- Para que o neurônio seja corretamente ajustado, pode-se ajustar os pesos e o threshold da função
- Ao definir o Bias como o threshold da função, pode-se usar a mesma fórmula de ajuste dos demais pesos.
- O valor 1 ou -1 define apenas o sinal do threshold. O valor 0 significa que o threshold será 0.

Perceptrons

- ❑ O neurônio MP, do ponto de vista geométrico, pode ser interpretado como uma reta (2D), ou um plano (3D) ou ainda um hiperplano ($> 3D$), que é usado para separar duas categorias de dados distintas.



Perceptron

❑ Estado de ativação:

- ❖ $1 = \text{ativo}$
- ❖ $-1 = \text{inativo}$

❑ Função de ativação:

- ❖ $a_i(t + 1) = f(u_i(t))$

- ❖ $a_i(t + 1) = \begin{cases} -1, & \text{se } u_i(t) < 0 \\ +1, & \text{se } u_i(t) \geq 0 \end{cases}$

$$u = \sum_{j=1}^N x_j w_j - \theta$$

Bias = -1
 $-1w_0$

Perceptrons

- ❑ Função de saída = função identidade
- ❑ Duas camadas:
 - ❖ Camada de pré-processamento
 - **M** máscaras fixas para extração de características
 - Podem implementar qualquer função, mas pesos são fixos.
 - ❖ Camada de discriminação (processamento):
 - Uma unidade de saída para discriminar padrões de entrada.
 - Pesos determinados através de aprendizado

Algoritmo de Treinamento Perceptron

1) Iniciar todas as conexões com $w_{ij} = 0$;

2) Repita

Para cada par de treinamento (X, d)

Calcular a saída y

Se $(y_d \neq y)$

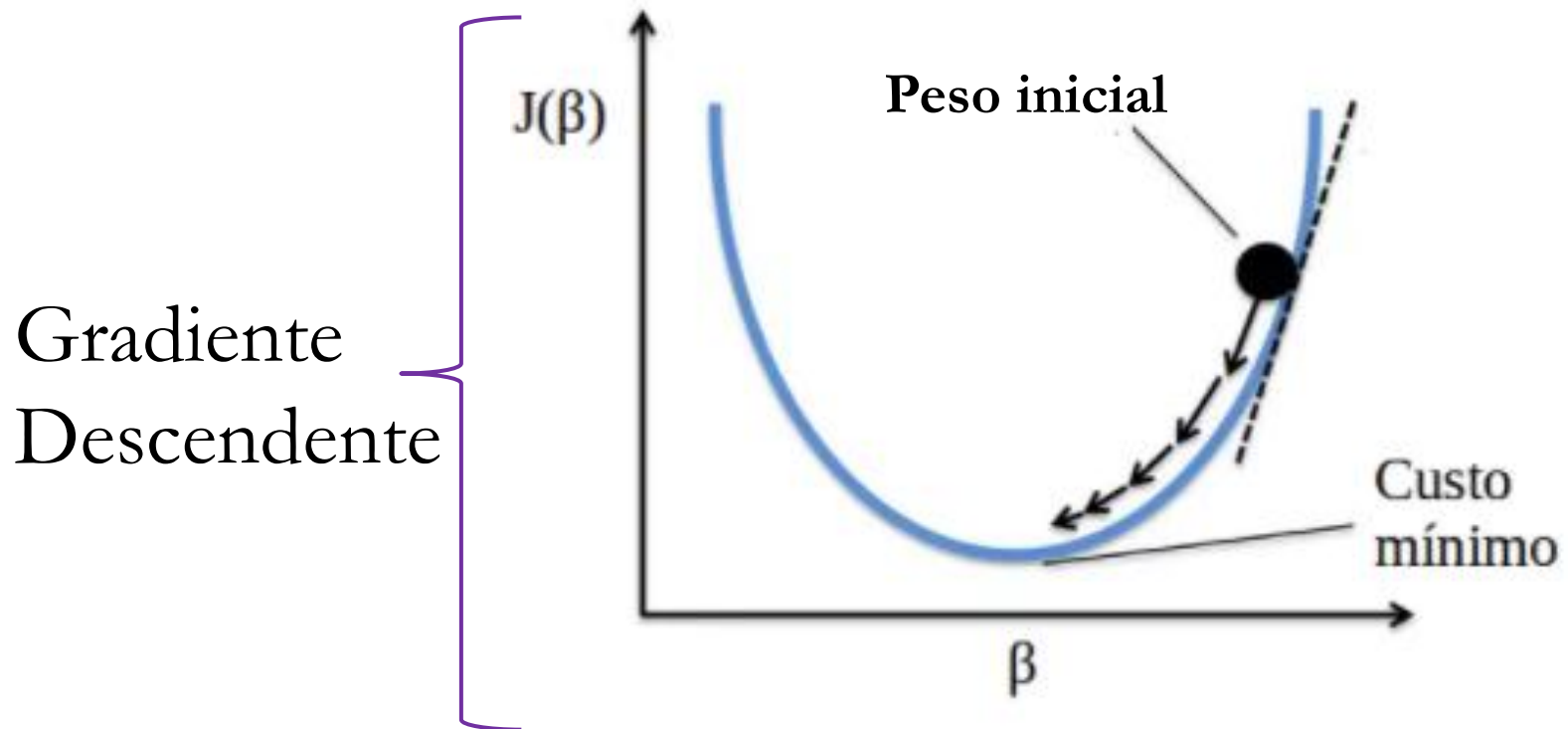
Então

Atualizar o vetor de pesos para cada um dos nós da rede
segundo a regra

$$w_i(t+1) = w_i(t) + \eta(y_d - y)x_i$$

Até o erro ser aceitável

Algoritmo de Treinamento Perceptron



Aprendizado por Correção de Erro

□ Taxa de aprendizado (η)

❖ $0 < \eta < 1$

❖ Taxas pequenas

Média das entradas anteriores

Estimativas através de peso

Aprendizado lento

❖ Taxas grandes

Aprendizado rápido

Captação de mudanças no processo

Instabilidade

Perceptrons e Adalines

- ❑ Poder Computacional: apenas funções **linearmente separáveis** (Minsky e Papert, 1969).

Problema do XOR



Perceptrons e Adalines

❑ Problemas Linearmente Separáveis:

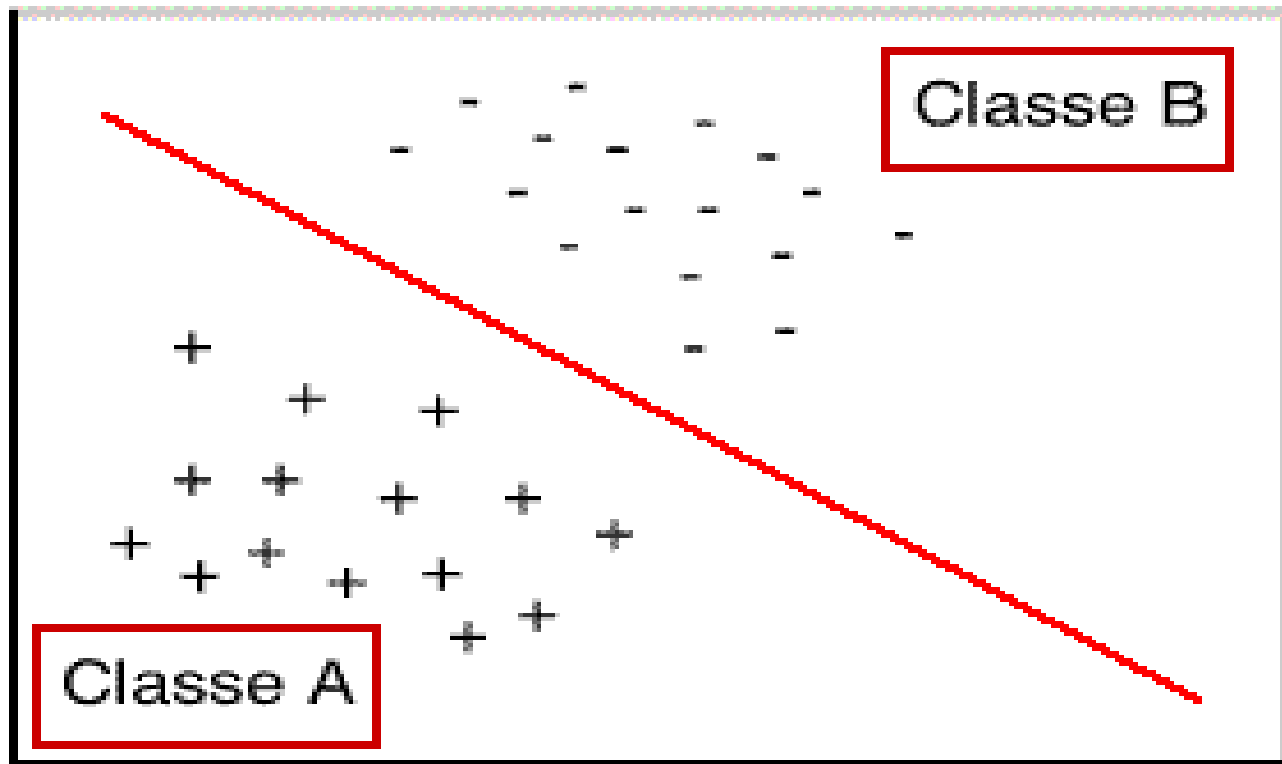


Fig. 1: Classes linearmente separáveis

Perceptrons e Adalines

- ❑ Problemas Não-Linearmente Separáveis:

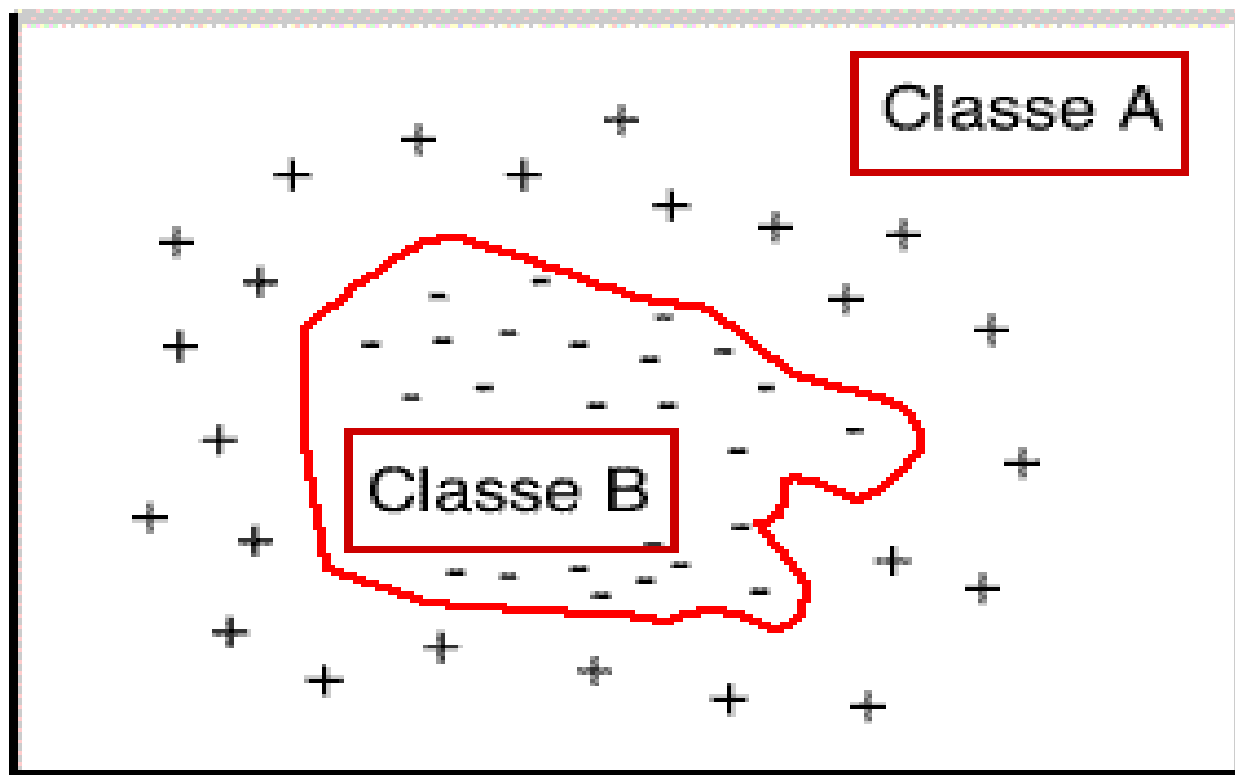
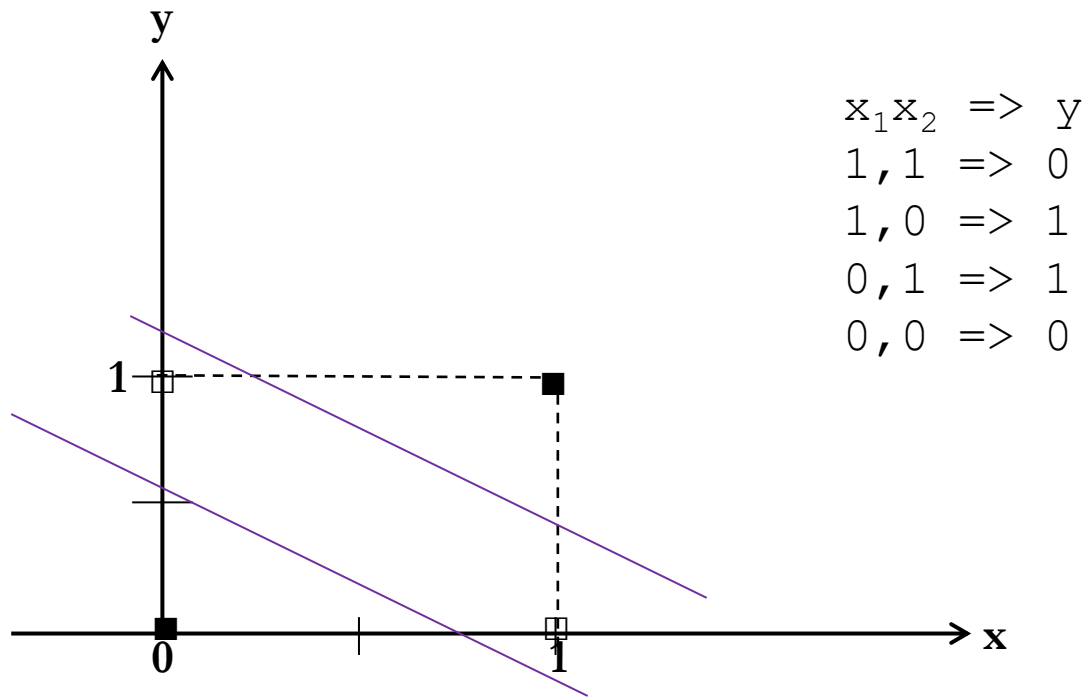


Fig. 2: Classes não linearmente separáveis

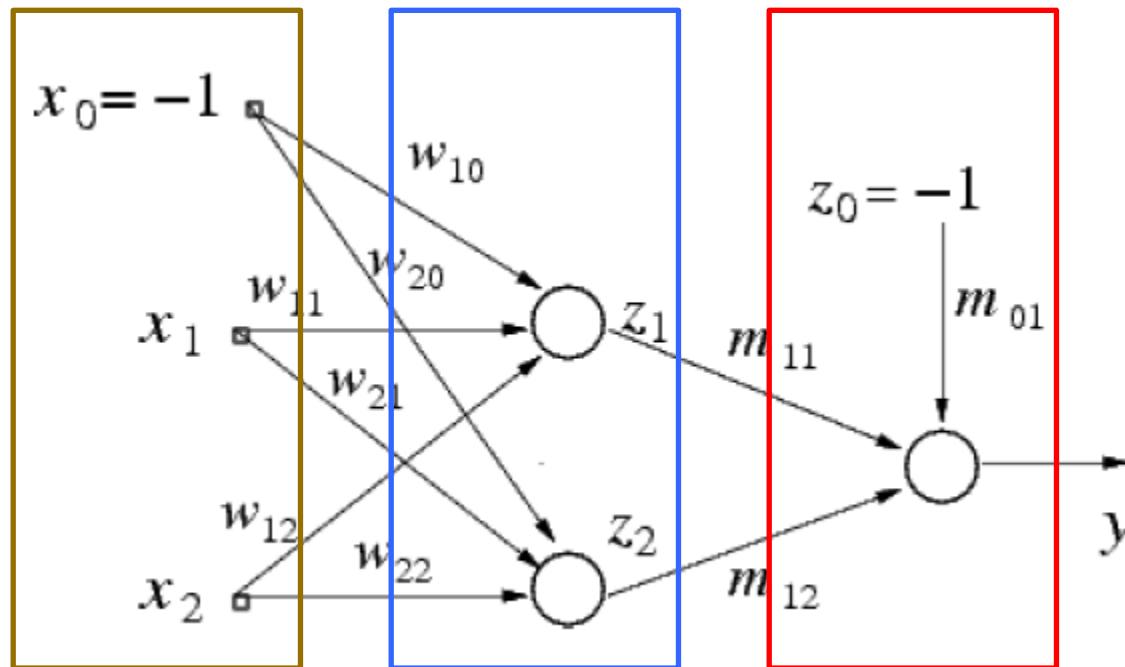
Perceptrons e Adalines

❑ Problema XOR Lógico:



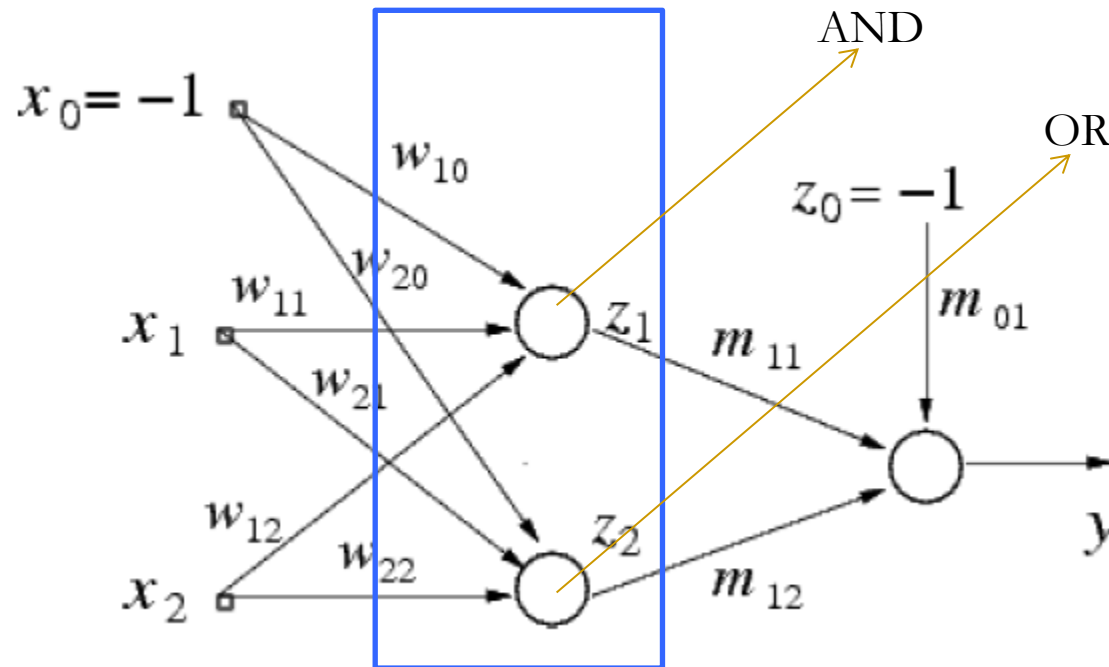
Perceptrons – Nova Proposta

- ❑ Para resolver o problema do **XOR** seriam necessários três neurônios.



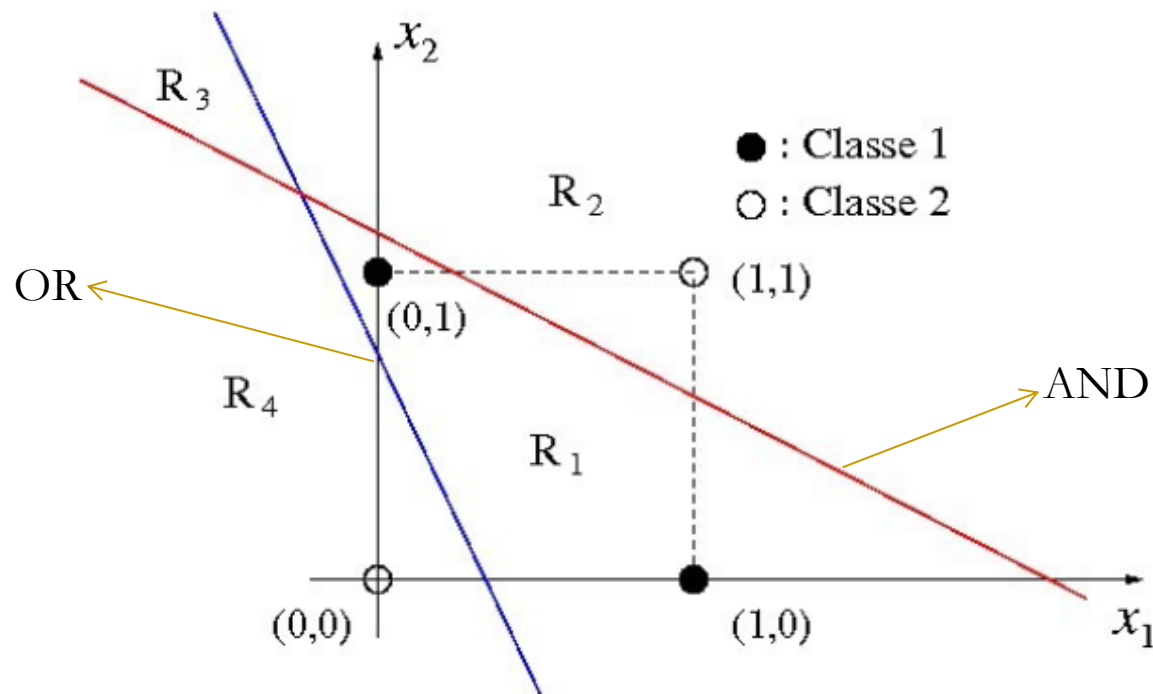
Perceptrons – Nova Proposta

- ❑ Para resolver o problema do **XOR** seriam necessários três neurônios.



Perceptrons – Nova Proposta

Exemplo (cont.) : Dois neurônios são necessários para separar o espaço (x_1, x_2) em 4 regiões (R_1, R_2, R_3, R_4).



Perceptrons – Nova Proposta

Exemplo (cont.) : Note que a reta em vermelho corresponde a um neurônio que implementa a porta AND, enquanto a reta em azul corresponde a um neurônio que implementa a porta OR.

Sejam z_1 e z_2 , as saídas dos neurônios responsáveis pelas retas em vermelho e azul, respectivamente. Assim, temos que:

Em R_1 , $z_1 = 0$ e $z_2 = 1$.

Em R_2 , $z_1 = 1$ e $z_2 = 1$.

Em R_3 , $z_1 = 1$ e $z_2 = 0$.

Em R_4 , $z_1 = 0$ e $z_2 = 0$.

Perceptrons – Nova Proposta

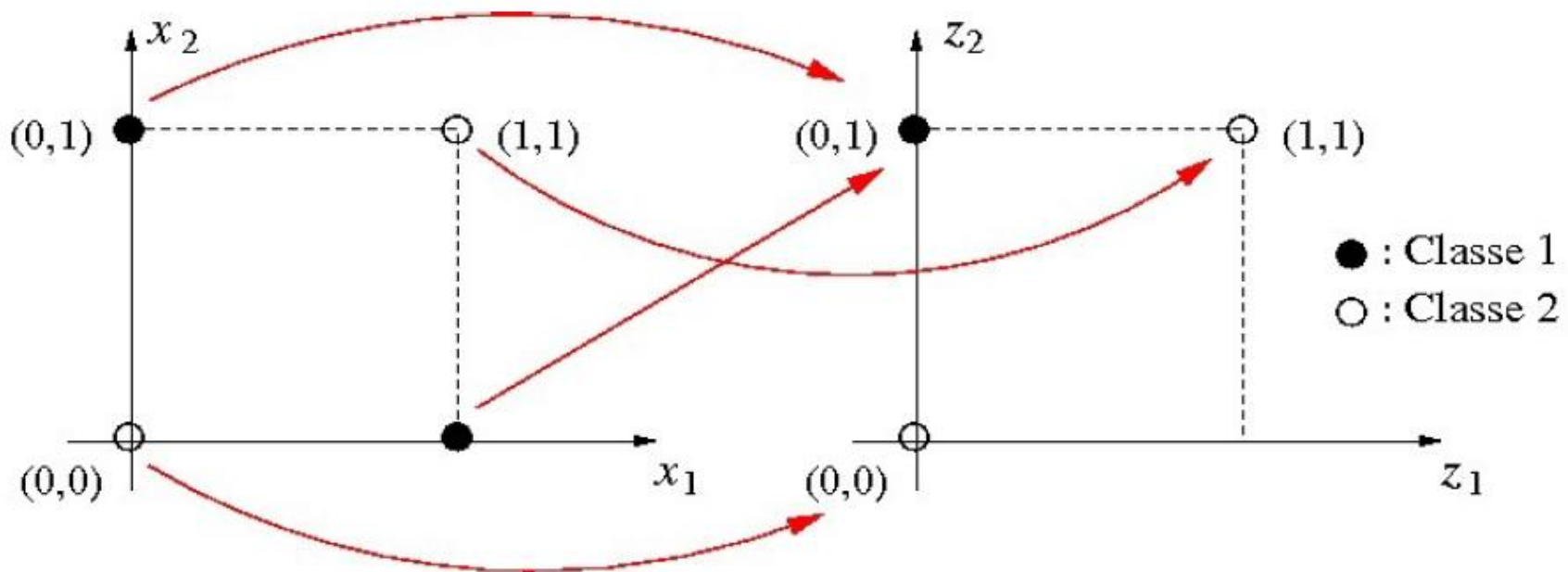
Exemplo (cont.) : Precisa-se ainda de um terceiro neurônio para combinar os valores de z_1 e z_2 , a fim de gerar o valor de y correto.

Quando um ponto da função que cair em qualquer uma das regiões R_2 , R_3 e R_4 , deve gerar uma saída igual a $y = 0$.

Quando um ponto da função que cair em na região R_1 , o terceiro neurônio deve gerar uma saída igual a $y = 1$.

Perceptrons – Nova Proposta

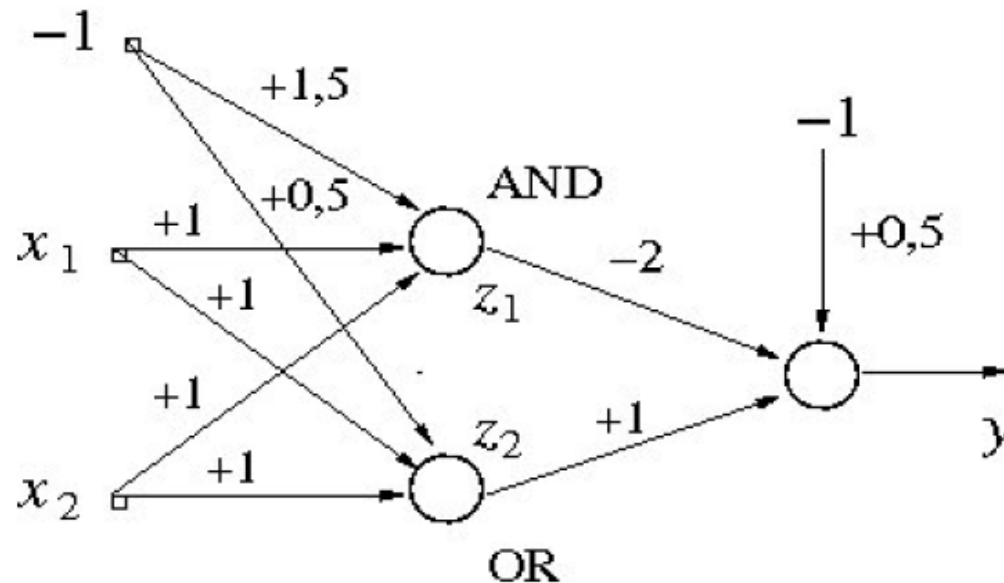
Exemplo (cont.) : Representação da função XOR no espaço (z_1, z_2) .



No espaço (z_1, z_2) a função XOR passa a ser lineamente separável, pois o ponto $(x_1, x_2) = (1,0)$ é mapeado no ponto $(z_1, z_2) = (0,1)$!

Adalines – Nova Proposta

Exemplo (cont.) : Colocando os dois primeiros neurônios em uma camada e o terceiro neurônio na camada seguinte (subseqüente), chega-se à seguinte rede multicamadas que implementa a porta XOR.

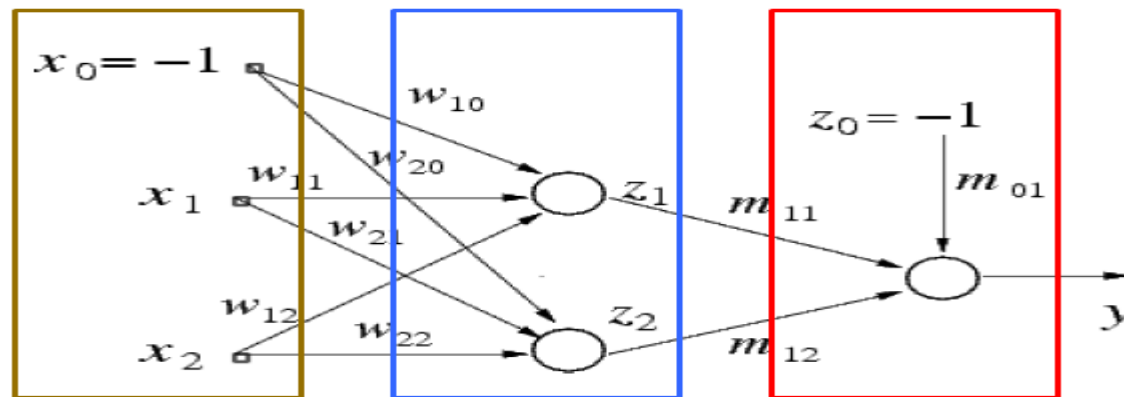


MLPs e Problemas não-lineares

- ❑ A não-linearidade pode ser obtida através da **função de ativação**.
- ❑ Objetivo: encontrar um método geral de aproximação de função não linear que consiga se ajustar a qualquer dado, independente de sua aparência.

Treinamento em MLP's

- ❑ Já vimos como treinar redes lineares (**Adalines e Perceptrons**) com o gradiente descendente.
- ❑ Se tentarmos usar o mesmo algoritmo para as MLPs encontraremos uma dificuldade, pois temos as saídas desejadas para as unidades da camada escondida.
- ❑ Como poderíamos ajustar os pesos destas unidades?



Backpropagation

- ❑ The backpropagation algorithm was originally introduced in the 1970s.
- ❑ But its importance wasn't fully appreciated until a famous 1986 paper by **David Rumelhart, Geoffrey Hinton, and Ronald Williams**.

<http://neuralnetworksanddeeplearning.com/chap2.html>

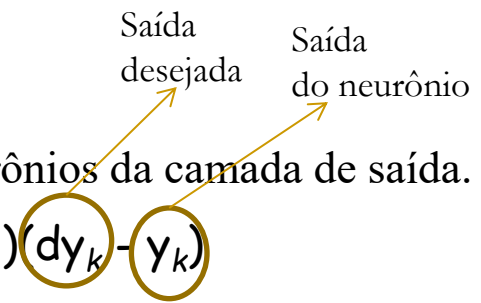
Backpropagation - o algoritmo (1/2)

1. Crie uma rede feed-forward com N_{in} neurônios de entrada N_h neurônios escondidos e N_{out} neurônios de saída
2. Inicialize os pesos da rede com valores aleatórios pequenos
3. Até que uma condição de término seja atingida, execute os próximos passos
 1. Para cada par (x,y) faça:

1. Produza a saída da rede

2. Calcule o erro

3. Calcule o termo do erro δ_k para os neurônios da camada de saída.

$$\delta_k \leftarrow y_k(1 - y_k)(dy_k - y_k)$$


4. Calcule o termo do erro δ_h para os neurônios da camada de escondida.

$$\delta_h \leftarrow y_h(1 - y_h) \sum_{k \in \text{outputs}} w_{kh} \delta_k$$

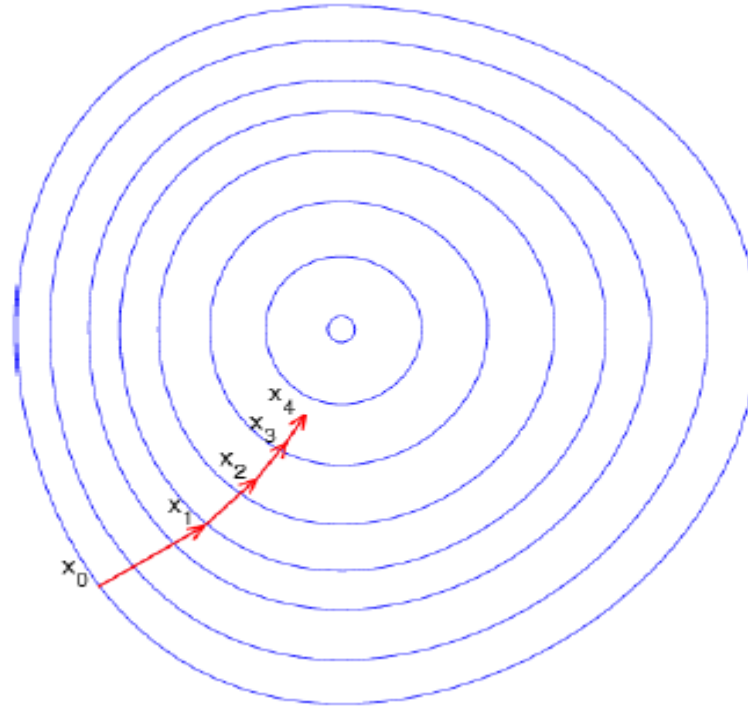
5. Atualize os pesos

$$w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$$

$$\Delta w_{ji} \leftarrow \eta \delta_j x_{ji}$$

Backpropagation - o algoritmo (2/2)

- Regra Delta $\Rightarrow$ Gradiente Descente
 - ❖ Método de otimização.



Gradiente Descendente

- ❑ Para entender, considere uma unidade **linear simples**

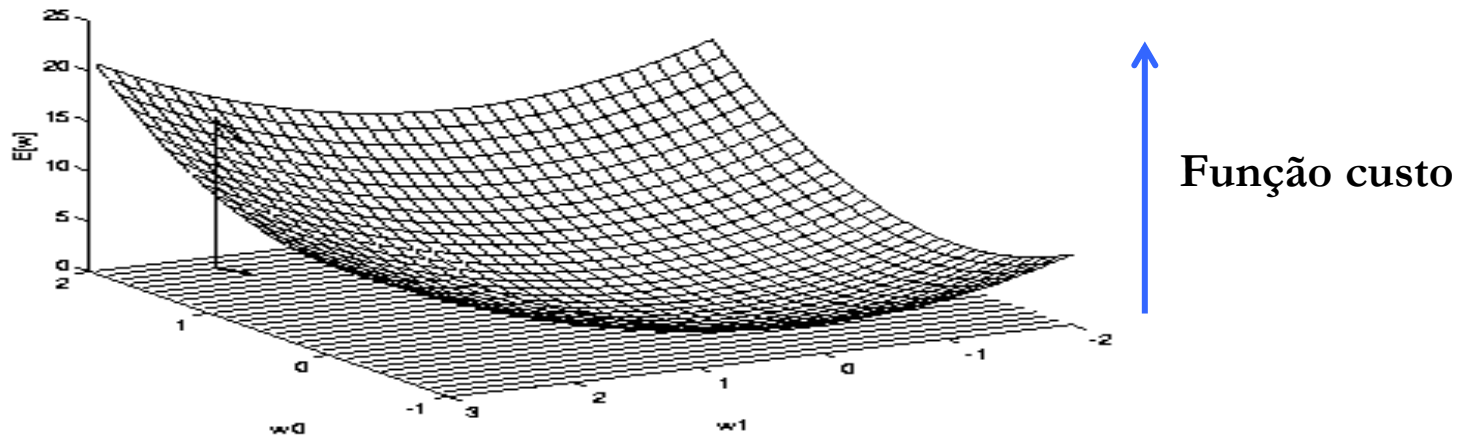
$$o = w_0 + w_1x_1 + \dots + w_nx_n$$

- ❑ Aprender w 's que minimize o erro quadrado

$$E[\vec{w}] \equiv \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2$$

- ❑ Onde D é o conjunto de treinamento

Gradiente Descendente



Gradient

$$\nabla E[\vec{w}] \equiv \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_n} \right]$$

Training rule:

$$\Delta \vec{w} = -\eta \nabla E[\vec{w}]$$

i.e.,

$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$$

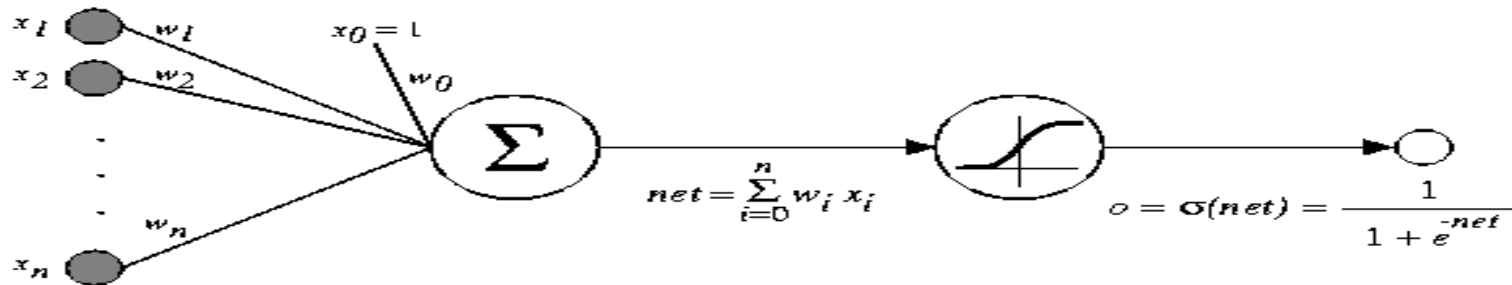
Erro

Gradiente Descendente

$$\begin{aligned}\frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i} \frac{1}{2} \sum_d (t_d - o_d)^2 \\ &= \frac{1}{2} \sum_d \frac{\partial}{\partial w_i} (t_d - o_d)^2 \\ &= \frac{1}{2} \sum_d 2(t_d - o_d) \frac{\partial}{\partial w_i} (t_d - o_d) \\ &= \sum_d (t_d - o_d) \frac{\partial}{\partial w_i} (t_d - \vec{w} \cdot \vec{x}_d) \\ \frac{\partial E}{\partial w_i} &= \sum_d (t_d - o_d) (-x_{i,d})\end{aligned}$$

Derivação
do Gradiente
Descendente
para uma função
Linear

Gradiente Descendente



$\sigma(x)$ is the sigmoid function

$$\frac{1}{1 + e^{-x}}$$

Nice property: $\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$

We can derive gradient decent rules to train

- One sigmoid unit
- *Multilayer networks* of sigmoid units $\rightarrow$ Backpropagation

Gradiente Descendente

$$\begin{aligned}\frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i} \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2 \\ &= \frac{1}{2} \sum_d \frac{\partial}{\partial w_i} (t_d - o_d)^2 \\ &= \frac{1}{2} \sum_d 2(t_d - o_d) \frac{\partial}{\partial w_i} (t_d - o_d) \\ &= \sum_d (t_d - o_d) \left(-\frac{\partial o_d}{\partial w_i} \right) \\ &= - \sum_d (t_d - o_d) \frac{\partial o_d}{\partial net_d} \frac{\partial net_d}{\partial w_i}\end{aligned}$$

But we know:

$$\frac{\partial o_d}{\partial net_d} = \frac{\partial \sigma(net_d)}{\partial net_d} = o_d(1 - o_d)$$

$$\frac{\partial net_d}{\partial w_i} = \frac{\partial (\vec{w} \cdot \vec{x}_d)}{\partial w_i} = x_{i,d}$$

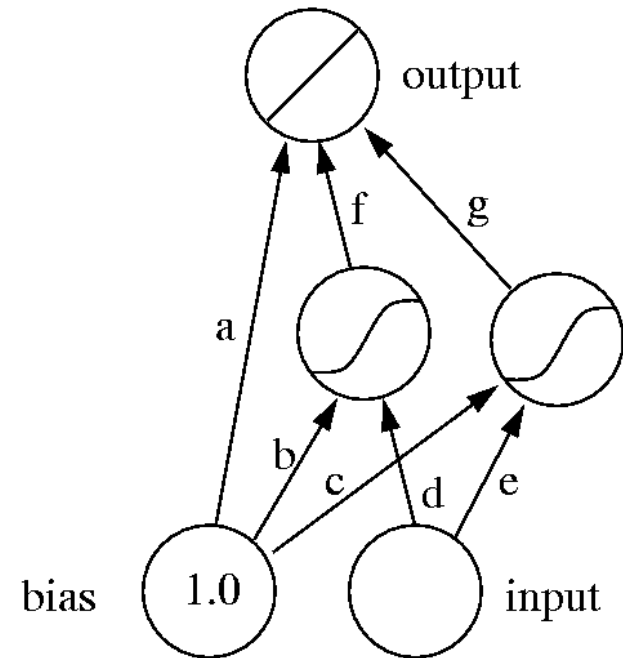
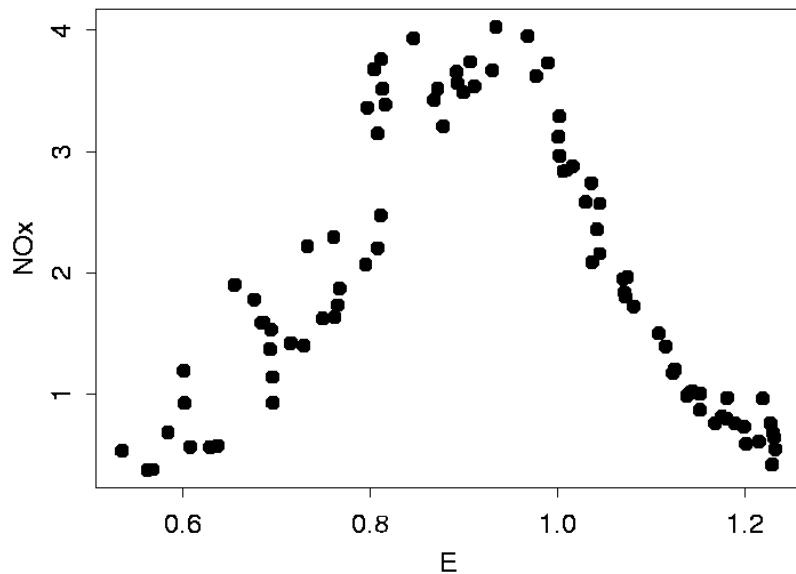
So:

$$\frac{\partial E}{\partial w_i} = - \sum_{d \in D} (t_d - o_d) o_d (1 - o_d) x_{i,d}$$

Derivação
do Gradiente
Descendente
para uma função
não Linear

Backpropagation - exemplo (1/9)

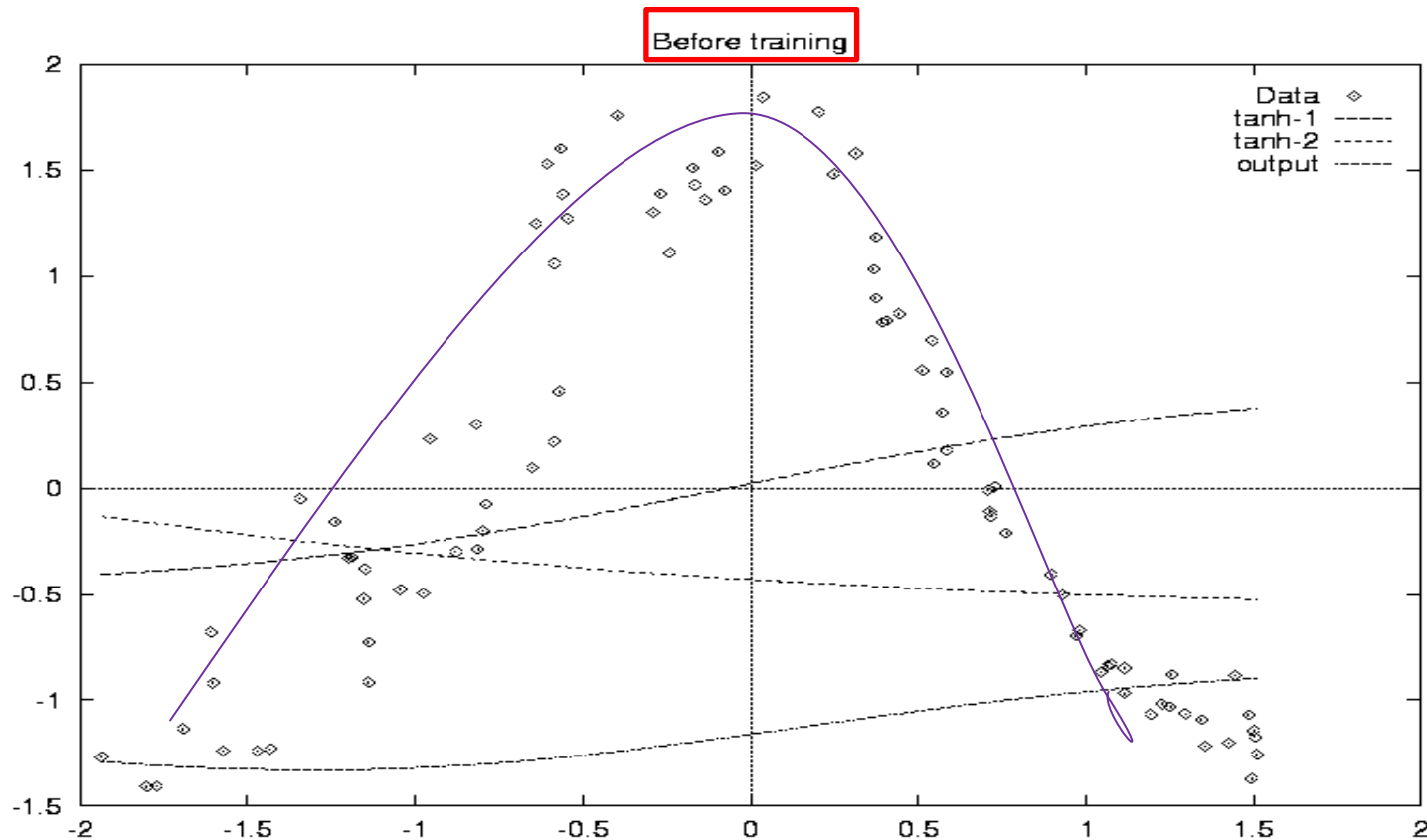
□ Demonstração de RNA como aproximador de função:



Backpropagation - exemplo (2/9)

- ❑ O nó de saída computa uma combinação de duas funções
- ❑ $y_o = f * \tanh_1(x) + g * \tanh_2(x) + a$, onde
- ❑ $\tanh_1(x) = \tanh(dx + b)$
- ❑ $\tanh_2(x) = \tanh(ex + c)$
- ❑ Inicializamos os pesos com valores aleatórios.
- ❑ Cada nó escondido computa uma função Tangente Hiperbólica.

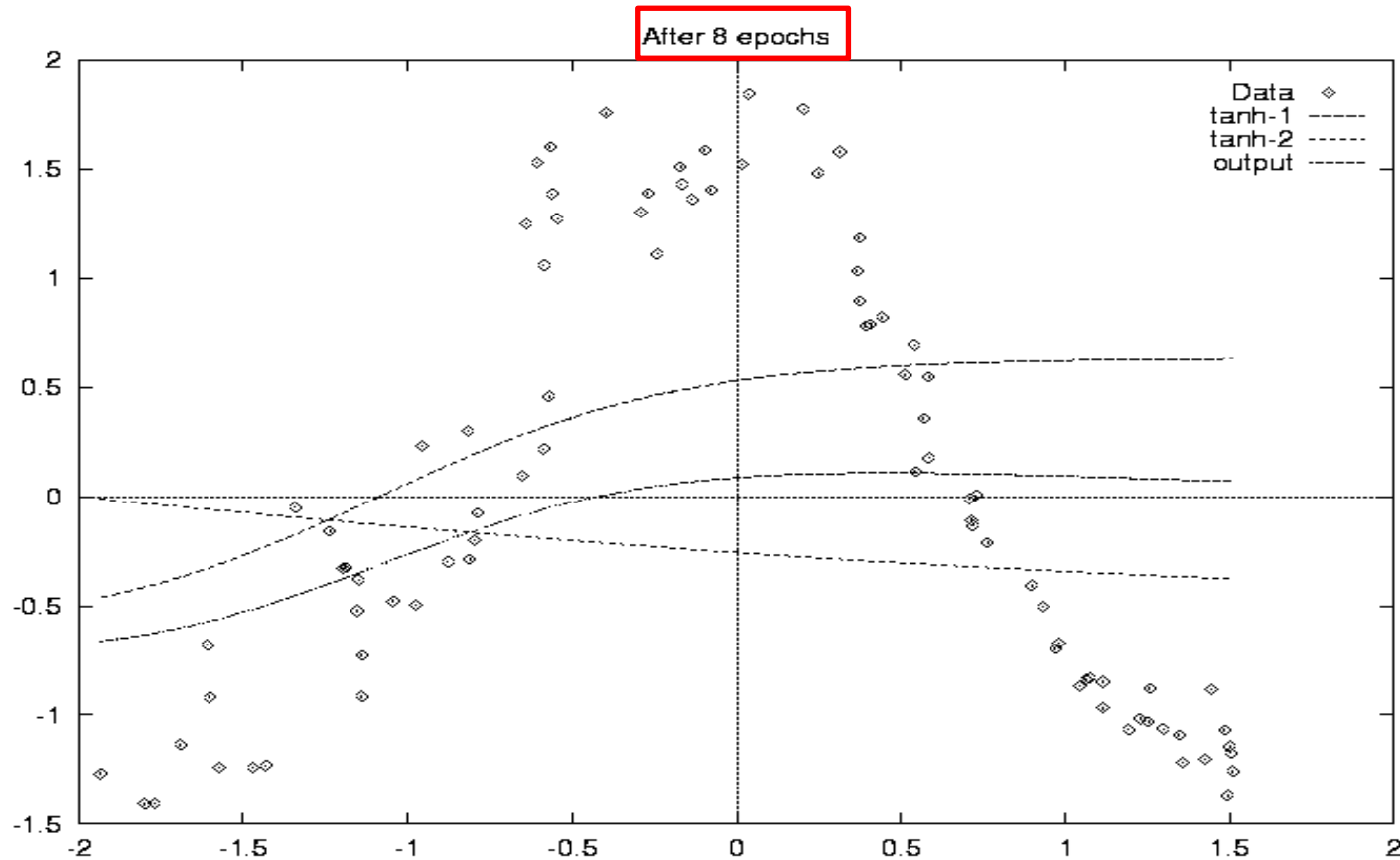
Backpropagation - exemplo (3/9)



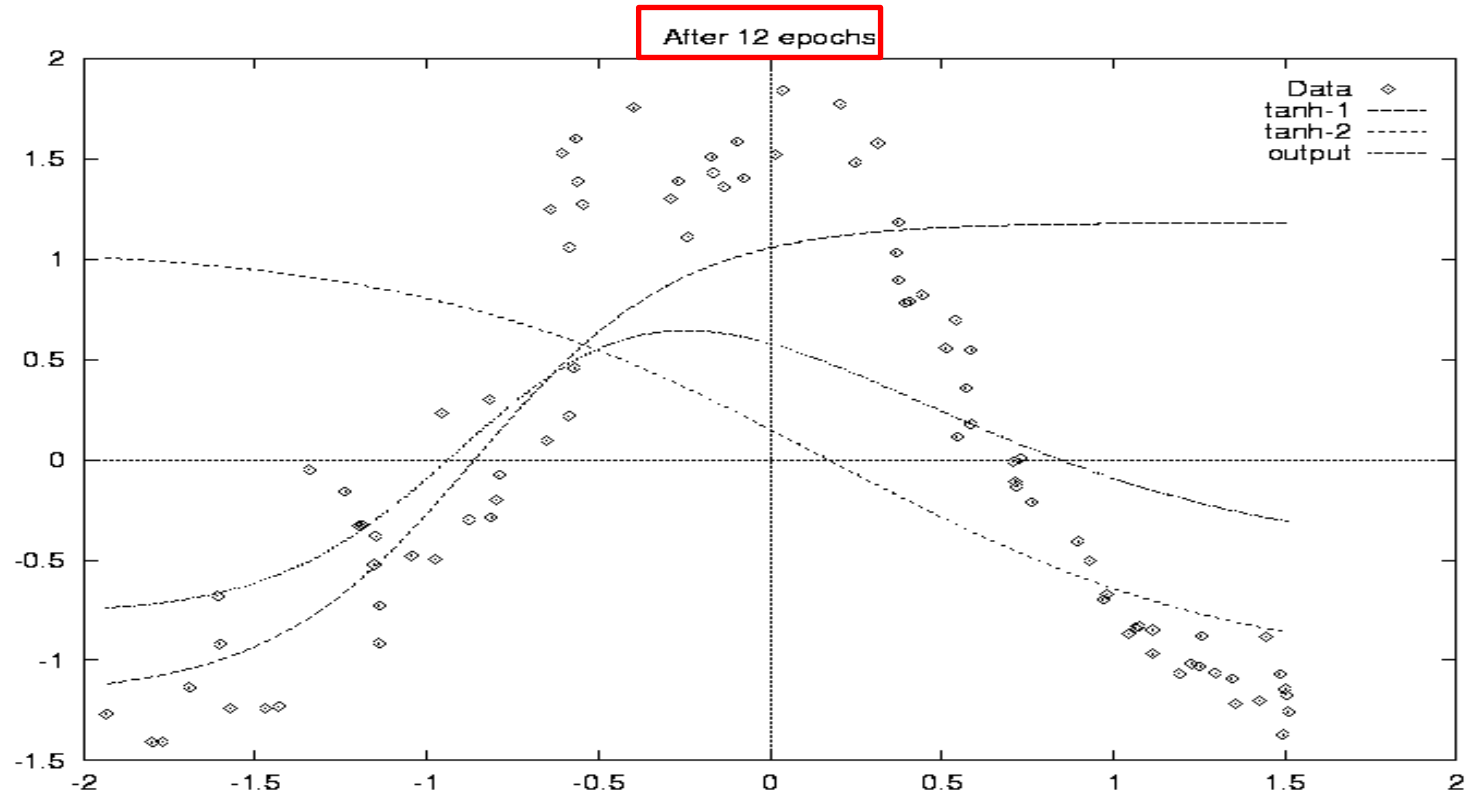
Backpropagation - exemplo (4/9)

- ❑ Treinamento da rede
- ❑ Taxa de aprendizado 0.3
- ❑ Atualização dos pesos após cada padrão
- ❑ Aprendizagem incremental
- ❑ Depois de passarmos por todo o conjunto de treinamento 20 vezes (20 épocas)
- ❑ As funções computadas pela rede têm a seguinte forma:

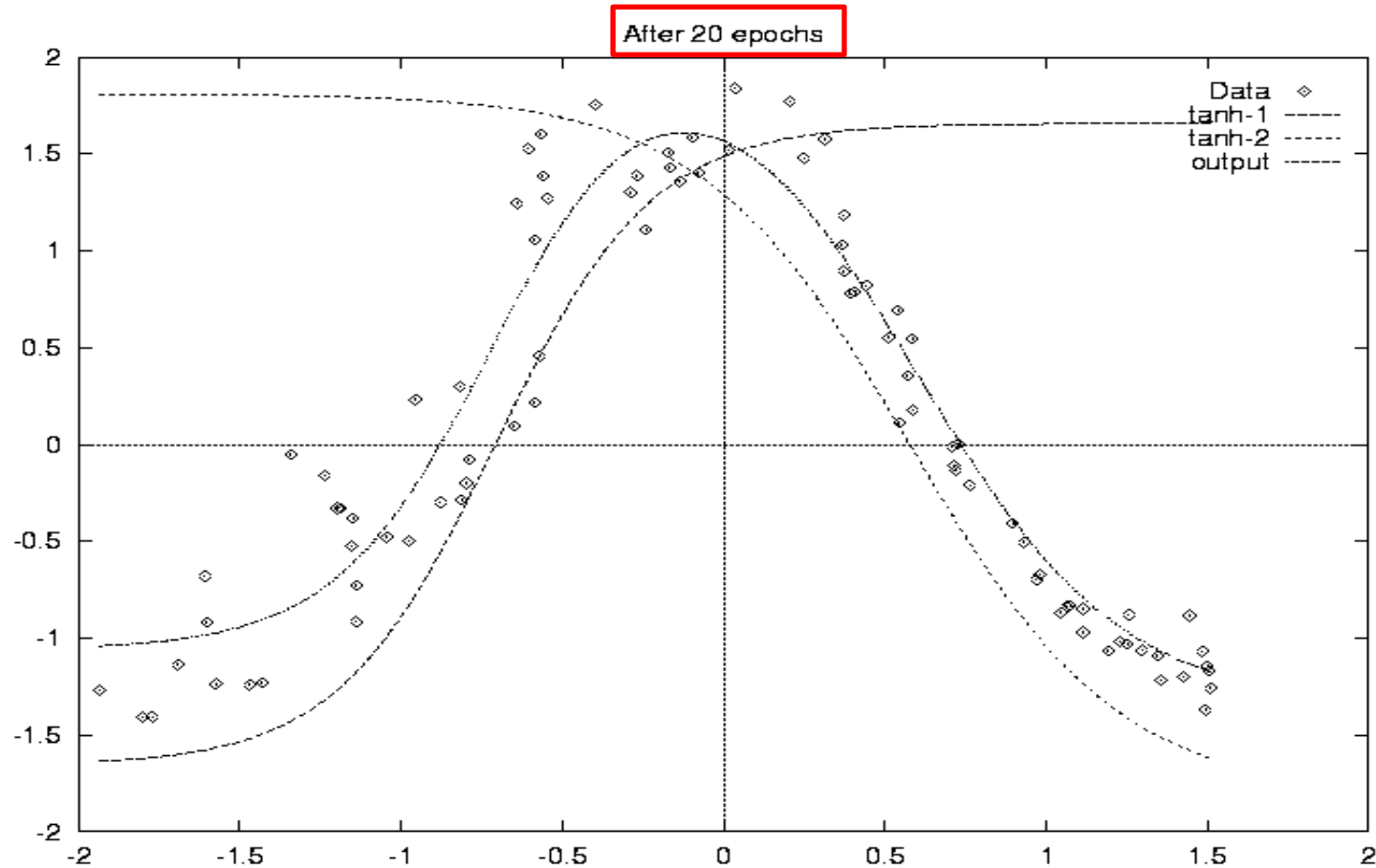
Backpropagation - exemplo (5/9)



Backpropagation - exemplo (6/9)



Backpropagation - exemplo (7/9)



MLPs e Camada Escondida

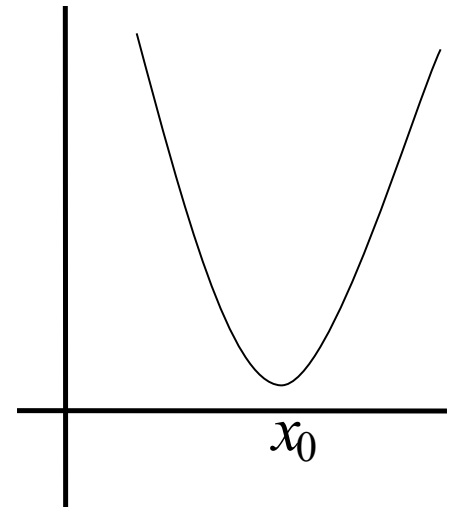
- **Teorema:** dado um número suficiente de nodos escondidos, uma MLP com apenas uma camada escondida pode aproximar **qualquer função contínua** (Cybenko, 1989).

Quantos???

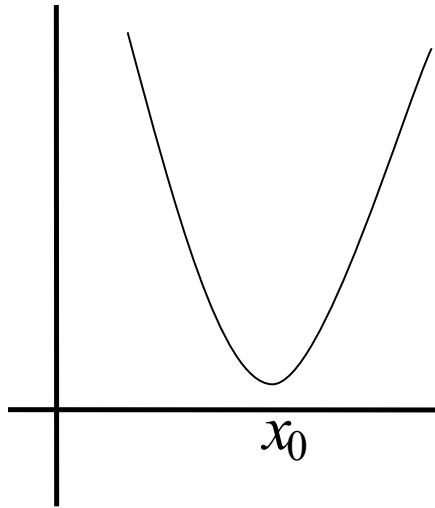
Backpropagation: um problema de otimização

- Achar o ponto de máximo ou mínimo de uma função $f(x, y, z, u, \dots)$ de n parâmetros, $x, y, z, u, \dots$

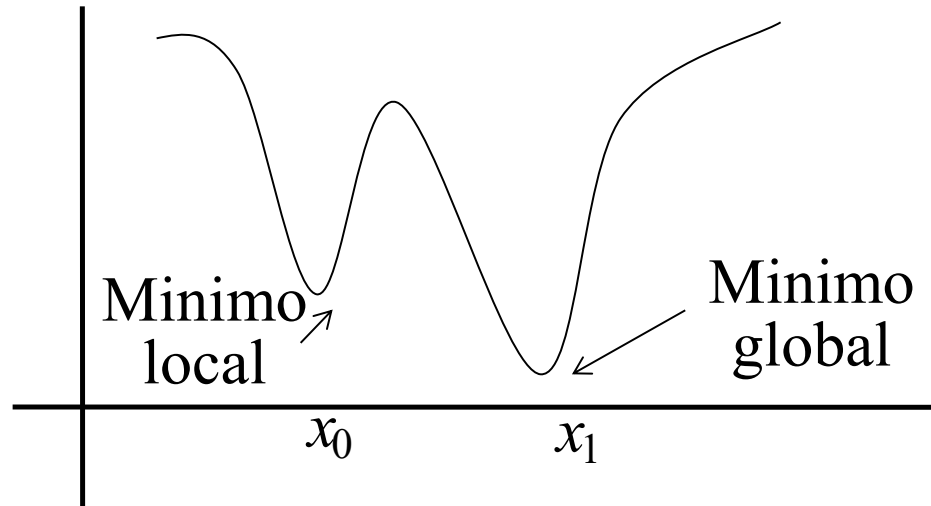
$$\text{minimizar } f(x) = \text{maximizar } -f(x)$$



Função Unimodal x Multimodal



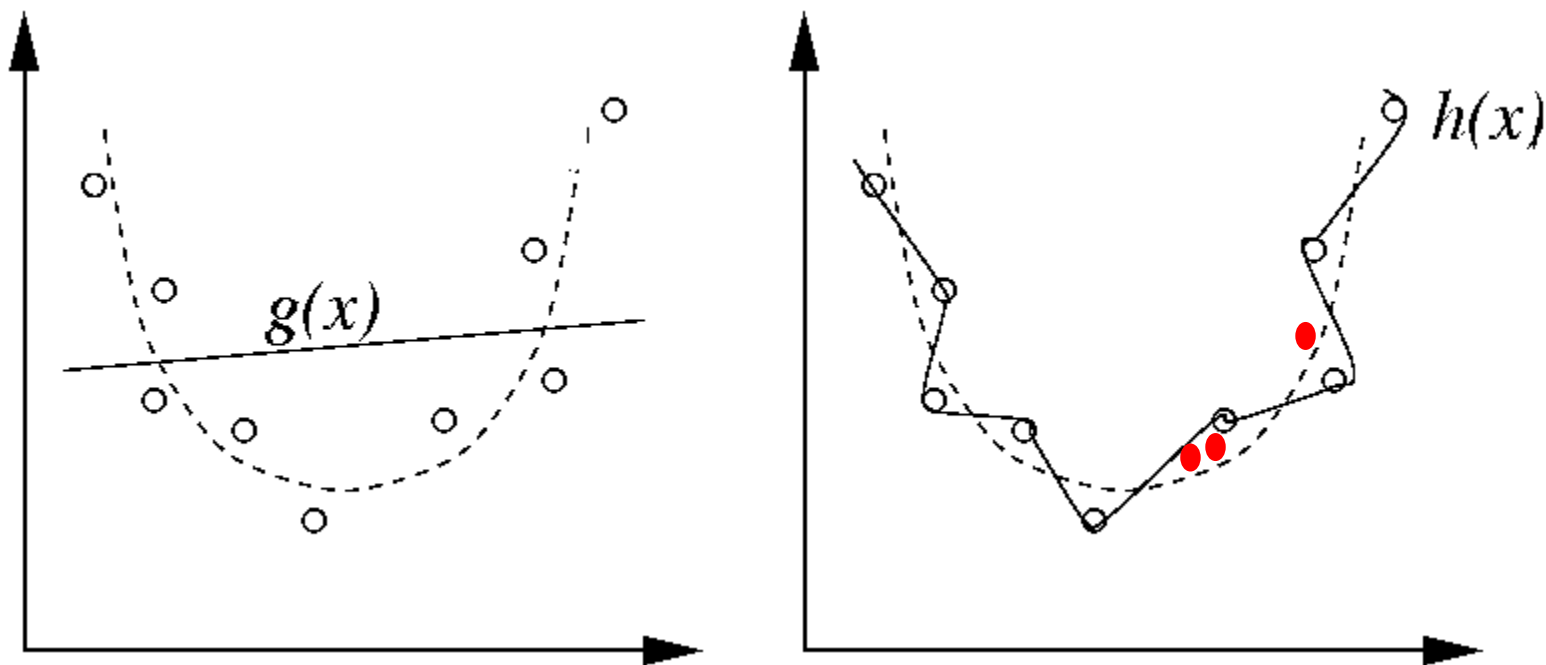
Função unimodal



Função multimodal

O Problema do Overfitting

- ❑ Na estrutura da Rede Neural:
 - ❖ Número de Neurônios (camada escondida)



O Problema do Overfitting: Estrutura

- ❑ No gráfico da esquerda, tentamos ajustar os pontos usando uma função $g(x)$ que tem poucos parâmetros: **uma reta**.
 - ❖ O Modelo é muito rígido e não consegue modelar nem o **conjunto de treinamento** nem **novos dados**.
 - ❖ O modelo têm um *bias* alto.
 - ❖ **Underfitting**.

O Problema do Overfitting: Estrutura

- ❑ O gráfico da direita mostra um modelo que foi ajustado usando muitos parâmetros livres.
 - ❖ Ele se ajusta **perfeitamente** ao dados do conjunto de treinamento.
 - ❖ Porém, este modelo não seria um bom “previsor” de $h(\mathbf{x})$ para novos valores de $\mathbf{x}$.
 - ❖ Dizemos que o modelo tem uma **variância** (*variance*) alta.
 - ❖ **Overfitting** (super-ajustamento).

O Problema do Overfitting: Estrutura

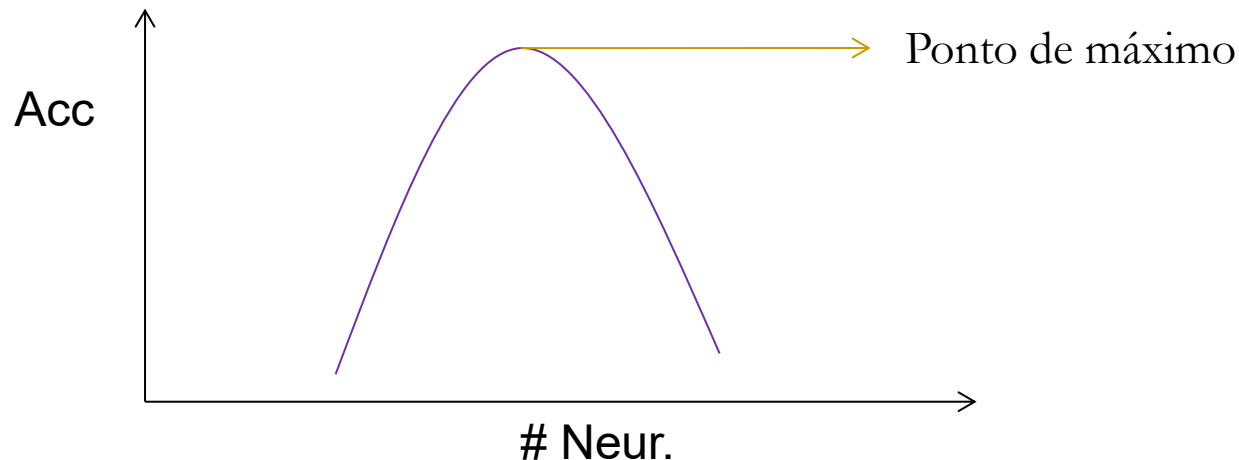
- Claramente, o que queremos é um compromisso entre:
 - ❖ um modelo que seja poderoso o bastante para representar a estrutura básica do dado $h(\mathbf{x})$,
 - ❖ Mas, não tão poderoso a ponto de modelar fielmente o ruído associado a um conjunto de treinamento.

Para base com 100 atributos e 4 valores de classe, então número de neurônios da camada escondida será igual a $(100 + 4) / 2 \Rightarrow 52$ neurônios.

O Problema do Overfitting: Estrutura

❑ Analisando:

Para base com 100 atributos e 4 valores de classe, então número de neurônios da camada escondida será igual a $(100 + 4) / 2 \Rightarrow 52$ neurônios.



O Problema do Overfitting

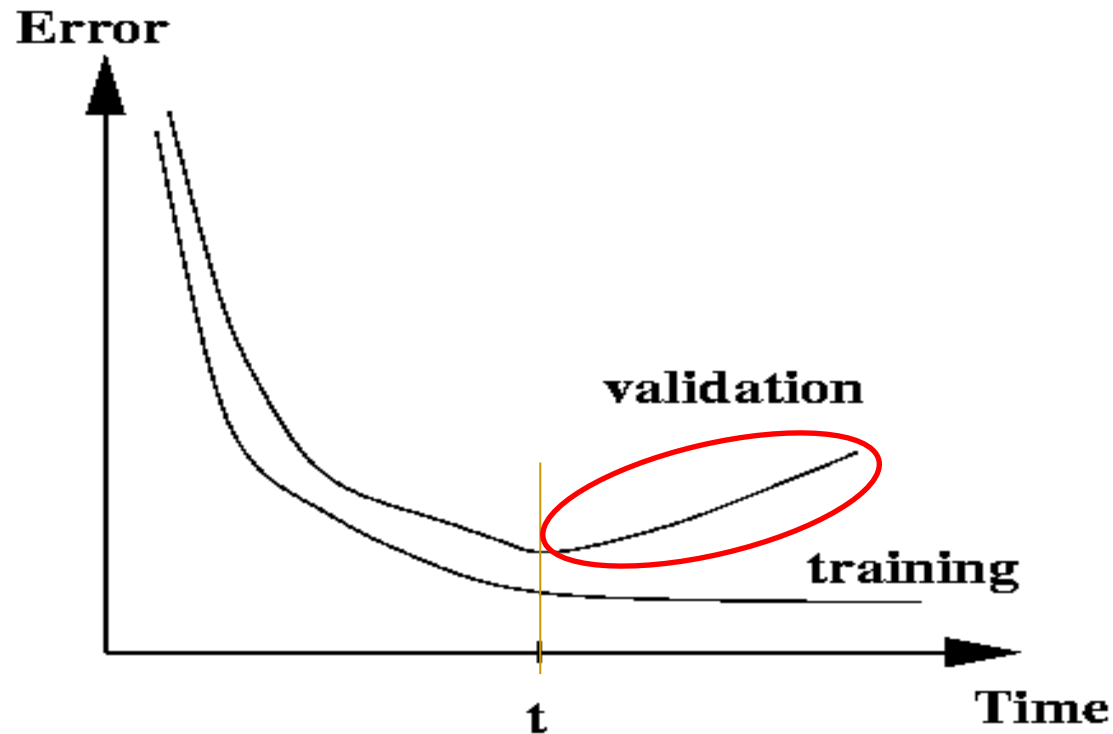
□ No processamento:

- ❖ Taxa de aprendizado; η
- ❖ Número de iterações.

$$\begin{aligned} \underline{w_{ji}} &\leftarrow \underline{w_{ji}} + \Delta \underline{w_{ji}} \\ \Delta \underline{w_{ji}} &\leftarrow \eta \delta_j \underline{x_{ji}} \end{aligned}$$

Prevenção do Overfitting

❑ Early stopping (técnica):



Análise do MLP

❑ Potencialidades:

- ❖ Habilidade de tratar sistemas complexos;
- ❖ Representação de conhecimento quantitativo;
- ❖ **Processamento Paralelo** (mesma camada);
- ❖ Aprendizado;
- ❖ Adaptabilidade;
- ❖ Generalização.

Análise do MLP

❑ Limitações:

- ❖ “Maldição” da dimensionalidade (# de neurônios);
- ❖ Overfitting;
- ❖ Alta complexidade computacional do treino;
- ❖ Mínimos locais.

Dúvidas ...

